

# **BEACON: A Behavior-Aware Software Layer for Electric-Vehicle Battery Health Monitoring from Voltage, Current and Time**

**A PROJECT REPORT**

*Submitted by*

**V NAVEEN**

**(CH.EN.U4CCE23054)**

**SANJAY P**

**(CH.EN.U4CCE23042)**

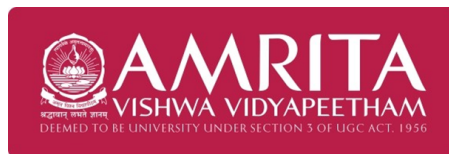
*in partial fulfillment for the award of the degree of*

**BACHELOR OF TECHNOLOGY IN COMPUTER AND  
COMMUNICATION ENGINEERING**

*Under the guidance of*

**Dr. M GAYATHRI**

*Submitted to*



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING  
AMRITA SCHOOL OF ENGINEERING  
AMRITA VISHWA VIDYAPEETHAM  
CHENNAI – 601103**

**October 2026**

# **BEACON: A Behavior-Aware Software Layer for Electric-Vehicle Battery Health Monitoring from Voltage, Current and Time**

**A PROJECT REPORT**

*Submitted by*

**V NAVEEN**

**(CH.EN.U4CCE23054)**

**SANJAY P**

**(CH.EN.U4CCE23042)**

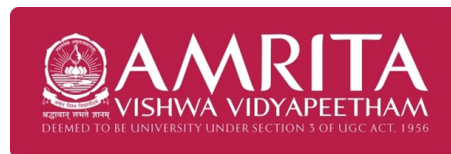
*in partial fulfillment for the award of the degree of*

**BACHELOR OF TECHNOLOGY IN COMPUTER AND  
COMMUNICATION ENGINEERING**

*Under the guidance of*

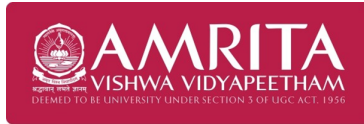
**Dr. M GAYATHRI**

*Submitted to*



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING  
AMRITA SCHOOL OF ENGINEERING  
AMRITA VISHWA VIDYAPEETHAM  
CHENNAI – 601103**

**October 2026**



**SCHOOL OF  
ENGINEERING  
CHENNAI**

## **BONAFIDE CERTIFICATE**

Certified that this project report **“BEACON: A Behavior-Aware Software Layer for Electric-Vehicle Battery Health Monitoring from Voltage, Current and Time”** is the bonafide work of **V NAVEEN (CH.EN.U4CCE23054), SANJAY P (CH.EN.U4CCE23042)** who carried out the project work under my supervision.

**SIGNATURE**

**Dr. LAKSHMANAN S A  
CHAIRPERSON**

Associate Professor  
Department of ECE,  
Amrita School of Engineering,  
Amrita Vishwa Vidyapeetham, Chennai.

**SIGNATURE**

**Dr. M GAYATHRI  
SUPERVISOR**

[DESIGNATION]  
Department of ECE,  
Amrita School of Engineering,  
Amrita Vishwa Vidyapeetham, Chennai.

The Viva-Voce Examination for Project Phase-I was conducted on \_\_\_\_\_

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**



SCHOOL OF  
ENGINEERING  
CHENNAI

## DECLARATION BY THE CANDIDATE

We declare that the report entitled “**BEACON: A Behavior-Aware Software Layer for Electric-Vehicle Battery Health Monitoring from Voltage, Current and Time**” submitted by us for the degree of BACHELOR OF TECHNOLOGY IN COMPUTER AND COMMUNICATION ENGINEERING is the record of the project work carried out by us under the guidance of **Dr. M GAYATHRI** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title in this or any other University or institution of higher learning.

**SIGNATURE**

**V NAVEEN**  
**CH.EN.U4CCE23054**

**SIGNATURE**

**SANJAY P**  
**CH.EN.U4CCE23042**

## ABSTRACT

A battery management system (BMS) measures terminal voltage, current and temperature, but whether software above it can turn these into a trustworthy state of health (SOH) and remaining useful life (RUL) is usually answered by fitting a model. This project builds BEACON, a three-tier telemetry and health-monitoring system, and tests that question with measured evidence. BEACON ingests data from a CAN bus, a USB serial bench rig or research datasets, converges all three on one schema, and scores them through one shared pipeline that is served over a REST API to a React dashboard. On 31 NASA cells in nine protocol cohorts, twelve published methods all lost skill under leave-one-cohort-out validation, so the system's two working estimators fit nothing across cells. SOH is the ratio of charge delivered within a fixed voltage window to the same quantity on the cell's own first discharges, with ohmic sag corrected by a resistance identified from the voltage step at each load onset. From voltage, current and time alone, on 22 CALCE lithium cobalt oxide cells, the median per-cell error was 1.7% (95% CI 1.4–4.3%) on 17 cells, with five refused for stated causes, and the step resistance tracked the cyclers' resistance on 19 of 20 cells. Unchanged, the estimator measured all eight cells of an independent Oxford dataset at 3.0% median error. RUL extrapolated from each cell's own fade was within  $\pm 20$  cycles 73% of the time in the last 25 cycles before 90% capacity. The system refuses to output a number it cannot justify: every output is validated, banded by its measured error, or refused with a reason. A bench rig built on a NodeMCU ESP8266 with an INA219 and an LM35 streamed 83 of 83 checksummed frames at a measured 1.000 s cadence, which validates data acquisition but not health estimation.

**Keywords:** battery management system, state of health, remaining useful life, electric vehicle, internal resistance, telemetry, uncertainty quantification

## ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives us immense pleasure to express our profound gratitude to our honorable Chancellor **SRI MATA AMRITANANDAMAYI DEVI**, for her blessings and for being a source of inspiration.

We are indebted to extend our gratitude to our Director, **Mr. I B MANIKANTAN**, Amrita Vishwa Vidyapeetham, Chennai for facilitating us all the facilities and extended support to gain valuable education and learning experience.

We register our special thanks to **Dr. V. JAYAKUMAR**, Principal, Amrita Vishwa Vidyapeetham, Chennai for the support given to us in the successful conduct of this project. We wish to express our sincere gratitude to **Dr. LAKSHMANAN S A**, Chairperson, Department of Electronics and Communication Engineering, and our supervisor **Dr. M GAYATHRI** for their inspiring guidance, personal involvement, and constant encouragement during the entire course of this work.

We are grateful to Project Coordinator, Review Panel Members and the entire faculty of the Department of Electronic & Communication Engineering, for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

**V NAVEEN**  
**CH.EN.U4CCE23054**

**SANJAY P**  
**CH.EN.U4CCE23042**

## **TABLE OF CONTENTS**

|                                                        |             |
|--------------------------------------------------------|-------------|
| <b>ABSTRACT</b>                                        | <b>iv</b>   |
| <b>ACKNOWLEDGEMENT</b>                                 | <b>v</b>    |
| <b>LIST OF TABLES</b>                                  | <b>vii</b>  |
| <b>LIST OF FIGURES</b>                                 | <b>viii</b> |
| <b>LIST OF SYMBOLS AND ABBREVIATIONS</b>               | <b>ix</b>   |
| <b>1 INTRODUCTION</b>                                  | <b>1</b>    |
| 1.1 BACKGROUND                                         | 1           |
| 1.2 MOTIVATION                                         | 1           |
| 1.3 PROBLEM STATEMENT                                  | 2           |
| 1.4 RESEARCH GAP                                       | 2           |
| 1.5 OBJECTIVES                                         | 3           |
| 1.6 KEY CONTRIBUTIONS                                  | 3           |
| 1.7 SCOPE AND ORGANISATION OF THE REPORT               | 4           |
| <b>2 LITERATURE REVIEW</b>                             | <b>5</b>    |
| 2.1 STATE ESTIMATION IN THE BATTERY MANAGEMENT SYSTEM  | 5           |
| 2.2 DATA-DRIVEN SOH ESTIMATION AND HOW IT IS VALIDATED | 5           |
| 2.3 DISCHARGE-CURVE ANALYSIS                           | 5           |
| 2.4 REMAINING USEFUL LIFE PROGNOSTICS                  | 6           |
| 2.5 PUBLIC DATASETS USED                               | 6           |
| 2.6 TELEMETRY ACQUISITION IN VEHICLES AND ON THE BENCH | 6           |
| 2.7 POSITIONING OF THIS WORK                           | 6           |
| 2.8 SUMMARY                                            | 7           |
| <b>3 METHODOLOGY / SYSTEM DESIGN</b>                   | <b>8</b>    |
| 3.1 PROBLEM AND DESIGN OBJECTIVE                       | 8           |
| 3.1.1 Research Variables                               | 8           |
| 3.2 OVERALL SYSTEM FRAMEWORK                           | 9           |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 3.2.1    | One Scoring Path, Extracted Rather Than Copied      | 9         |
| 3.3      | DATASETS                                            | 9         |
| 3.4      | FROM TELEMETRY TO DISCHARGES                        | 10        |
| 3.5      | VOLTAGE-WINDOW STATE OF HEALTH                      | 10        |
| 3.6      | RESISTANCE FROM THE LOAD STEP AND OHMIC CORRECTION  | 11        |
| 3.7      | GATES                                               | 12        |
| 3.8      | LEARNED WINDOWS FOR PARTIAL DISCHARGES              | 12        |
| 3.9      | REMAINING USEFUL LIFE FROM THE CELL'S OWN FADE      | 12        |
| 3.10     | EVIDENCE SUFFICIENCY                                | 13        |
| 3.11     | BEHAVIOUR SCORING LAYER (HEURISTIC)                 | 13        |
| 3.12     | VALIDATION PROTOCOL                                 | 14        |
| 3.13     | EVALUATION METRICS                                  | 15        |
| 3.14     | EXPERIMENTAL CONTROLS AND REPRODUCIBILITY           | 15        |
| <b>4</b> | <b>IMPLEMENTATION AND DEPLOYMENT</b>                | <b>16</b> |
| 4.1      | IMPLEMENTATION OVERVIEW                             | 16        |
| 4.2      | DEVELOPMENT ENVIRONMENT AND TOOLS                   | 16        |
| 4.3      | CODE ORGANISATION                                   | 16        |
| 4.4      | TELEMETRY ACQUISITION: THREE TRANSPORTS, ONE FRAME  | 18        |
| 4.4.1    | The Seam Is Narrow                                  | 18        |
| 4.5      | SERIAL WIRE PROTOCOL                                | 18        |
| 4.5.1    | Framing by Sentinel                                 | 18        |
| 4.5.2    | Error Detection                                     | 19        |
| 4.5.3    | Channel Utilisation                                 | 19        |
| 4.6      | REFUSAL GATES                                       | 19        |
| 4.7      | HARDWARE BENCH RIG                                  | 21        |
| 4.7.1    | Requirement and Design Calculations                 | 21        |
| 4.7.2    | As-Built Rig                                        | 22        |
| 4.7.3    | Firmware                                            | 22        |
| 4.7.4    | Bring-Up Procedure                                  | 23        |
| 4.8      | SERVICE TIER: REST API, GATEWAY AND DASHBOARD       | 23        |
| 4.9      | HEALTH REPORT CARD                                  | 24        |
| 4.10     | TESTING, CONTINUOUS INTEGRATION AND REPRODUCIBILITY | 24        |
| 4.11     | CONTAINERISATION AND DEPLOYMENT                     | 25        |
| 4.12     | CHAPTER SUMMARY                                     | 25        |
| <b>5</b> | <b>RESULTS AND DISCUSSION</b>                       | <b>26</b> |
| 5.1      | FITTED MODELS DO NOT SURVIVE A CHANGE OF PROTOCOL   | 26        |
| 5.2      | FIELD SOH FROM VOLTAGE, CURRENT AND TIME            | 27        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 5.3      | PARTIAL DISCHARGES                              | 29        |
| 5.4      | RESISTANCE TRACKS THE INSTRUMENT                | 29        |
| 5.5      | REMAINING USEFUL LIFE                           | 29        |
| 5.6      | RUL WHEN COMPLETE DISCHARGES ARE RARE           | 30        |
| 5.7      | WHEN THE EVIDENCE SUFFICES                      | 31        |
| 5.8      | EXTERNAL VALIDATION ON A SECOND DATASET         | 31        |
| 5.9      | A THIRD DATASET AND KNOWING WHEN TO REFUSE      | 32        |
| 5.10     | SYSTEM AND HARDWARE RESULTS                     | 33        |
| 5.10.1   | Bench Rig Captures                              | 33        |
| 5.10.2   | Fault Injection                                 | 33        |
| 5.10.3   | What the Hardware Result Does and Does Not Show | 33        |
| 5.11     | A WORKED EXAMPLE: THE REPORT CARD               | 34        |
| 5.12     | DISCUSSION                                      | 34        |
| 5.13     | VALIDATION SUMMARY                              | 35        |
| <b>6</b> | <b>CONCLUSION AND FUTURE WORK</b>               | <b>37</b> |
| 6.1      | LIMITATIONS                                     | 38        |
| 6.2      | FUTURE WORK                                     | 38        |
|          | <b>REFERENCES</b>                               | <b>39</b> |
|          | <b>APPENDIX A</b>                               | <b>41</b> |
|          | <b>APPENDIX B</b>                               | <b>42</b> |

## LIST OF TABLES

|           |                                                                                                                                 |    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------|----|
| Table 2.1 | Approach Families and the Properties That Matter for a Deployed Layer                                                           | 7  |
| Table 3.1 | Data Sources and Their Role                                                                                                     | 10 |
| Table 3.2 | Risk Categories of the Rule-Based Layer                                                                                         | 13 |
| Table 3.3 | Health Index, Battery State and Digital-Twin State                                                                              | 14 |
| Table 4.1 | Software Components Used                                                                                                        | 17 |
| Table 4.2 | Organisation of the Core Package                                                                                                | 17 |
| Table 4.3 | Frame Size and Link Performance of the JSON Codec                                                                               | 19 |
| Table 4.4 | Gates and What Each Prevents                                                                                                    | 20 |
| Table 4.5 | Shunt Sizing for the INA219 (320 mV Full Scale, 10 $\mu$ V LSB)                                                                 | 21 |
| Table 4.6 | Design Document Versus As-Built Rig                                                                                             | 22 |
| Table 4.7 | Firmware Compilation Record ( <code>arduino-cli</code> , all warnings enabled)                                                  | 23 |
| Table 4.8 | Services                                                                                                                        | 23 |
| Table 5.1 | Benchmark on 31 NASA Cells in Nine Cohorts (Six of the Methods Shown); LOCO Intervals Are 95% Bootstrap Over Folds              | 26 |
| Table 5.2 | Feature Ablation and Shuffle Control (NASA SOH, LOCO)                                                                           | 27 |
| Table 5.3 | Field SOH at the 3.90–3.60 V Window; Median Per-Cell Absolute Error Against Cyclers Capacity, 95% Bootstrap Interval Over Cells | 27 |
| Table 5.4 | SOH from Real Partial Cycling (CALCE CS2 Types 5 and 6)                                                                         | 29 |
| Table 5.5 | RUL to 90% Capacity. Left: By True Cycles to End of Life. Right: By Predicted Cycles, With the 90% Range of (True – Predicted)  | 30 |
| Table 5.6 | RUL from Sparse Complete Discharges; None Is Accurate Enough to Report                                                          | 31 |
| Table 5.7 | Error by the Consistency Criterion of Eq. (3.7)                                                                                 | 31 |

|            |                                                                                                 |    |
|------------|-------------------------------------------------------------------------------------------------|----|
| Table 5.8  | External Validation on Eight Oxford Cells: Median and Worst<br>Per-Cell Mean Absolute SOH Error | 32 |
| Table 5.9  | Bench Rig Captures                                                                              | 33 |
| Table 5.10 | Status of the Main Outputs                                                                      | 36 |
| Table A.1  | Artifacts and Scripts Behind Each Results Section                                               | 41 |

## LIST OF FIGURES

|                                                                                                                                                                                                                                                                                             |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 3.1 Overall system framework: three transports, one unified frame, one scoring function, and the service tier.                                                                                                                                                                       | 9  |
| Figure 3.2 Field SOH estimator: from raw telemetry to a banded or refused output.                                                                                                                                                                                                           | 11 |
| Figure 4.1 A data frame: sentinel BEACON1, kind D, JSON body, and the XOR checksum *06, ending in a newline.                                                                                                                                                                                | 18 |
| Figure 4.2 As-built bench rig: measurement chain from the cell to the host.                                                                                                                                                                                                                 | 22 |
| Figure 5.1 SOH from voltage, current and time against every laboratory capacity test. Top: two constant-current cells, fixed window (mean error 0.9 and 1.1 points). Bottom: two cells of real partial cycling, learned window, estimate from partial discharges only (3.4 and 3.2 points). | 28 |
| Figure 5.2 Mean absolute SOH error for every measured CALCE cell, field method; the dashed line is the median (1.7%).                                                                                                                                                                       | 28 |
| Figure 5.3 RUL to 90% capacity. Left: predicted against actual cycles left in the last 50 cycles. Right: share of estimates within $\pm 20$ cycles by actual distance to end of life.                                                                                                       | 30 |
| Figure 5.4 Abridged health report card for CS2_35 at cycle 90, with the laboratory check.                                                                                                                                                                                                   | 34 |

## LIST OF SYMBOLS AND ABBREVIATIONS

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <b>ADR</b>        | - Architecture Decision Record                                |
| <b>API</b>        | - Application Programming Interface                           |
| <b>BMS</b>        | - Battery Management System                                   |
| <b>CAN</b>        | - Controller Area Network                                     |
| <b>CC/CV</b>      | - Constant Current / Constant Voltage                         |
| <b>CI</b>         | - Confidence Interval (or Continuous Integration, by context) |
| <b>DBC</b>        | - CAN database file describing signal layout                  |
| <b>EV</b>         | - Electric Vehicle                                            |
| <b>LCO</b>        | - Lithium Cobalt Oxide                                        |
| <b>LOBO</b>       | - Leave-One-Battery (cell)-Out validation                     |
| <b>LOCO</b>       | - Leave-One-Cohort-Out validation                             |
| <b>LRC</b>        | - Longitudinal Redundancy Check (XOR checksum)                |
| <b>MAE</b>        | - Mean Absolute Error                                         |
| <b>NMC</b>        | - Lithium Nickel Manganese Cobalt Oxide                       |
| <b>OCV</b>        | - Open-Circuit Voltage                                        |
| <b>RUL</b>        | - Remaining Useful Life                                       |
| <b>SOC</b>        | - State of Charge                                             |
| <b>SOH</b>        | - State of Health                                             |
| $R_{\text{step}}$ | - Apparent resistance identified from the load step           |

# CHAPTER 1

## INTRODUCTION

### 1.1 BACKGROUND

A battery management system (BMS) protects a lithium-ion pack and estimates its state of charge. It already holds the three measurements from which ageing is visible: terminal voltage, current and temperature [1]. A software layer above it can, in principle, turn those measurements into answers that an operator or driver can act on: how much of its original capacity the battery retains (state of health, SOH), how fast that is changing, how long it has before it should be replaced (remaining useful life, RUL), and what usage is doing to it.

Capacity and resistance are the two quantities through which ageing usually appears. Capacity fades as lithium inventory and active material are lost, and internal resistance grows, which reduces the power the cell can deliver [3, 4]. Reviews of SOH estimation distinguish direct measurement, model-based estimation and data-driven approaches [2, 1]. A vehicle, however, rarely provides the conditions a laboratory does: discharges are partial, rates vary, and no reference test is available. Any practical health estimator must therefore work from what the vehicle actually records.

### 1.2 MOTIVATION

The dominant way to build such a software layer is to fit a model. Features are extracted from cycling data and a regressor is trained to predict SOH, with feature sets ranging from usage aggregates to discharge-curve descriptors such as the capacity–voltage difference between cycles [6]. Skill is commonly reported under leave-one-cell-out validation, in which a held-out cell’s siblings from the same test protocol remain in the training set, so the test asks whether the model recognises a protocol it has already seen.

A deployed BMS faces a harder question: the next battery comes from conditions the training set never contained. This project began as a battery-degradation study built on fitted models, and measured that question directly. On a benchmark of 31 NASA cells in nine protocol cohorts, every one of twelve published methods lost skill when the held-out unit changed from a cell to a whole protocol cohort, and no method could be distinguished from another (Chapter 5). That negative result is the motivation for what follows: if models fitted across cells do not transfer, a health layer for a vehicle has to be built from estimators that fit nothing across cells, and from a pipeline that is honest about when it does not know.

The work is also motivated by engineering practice. A monitoring system that is asked for a number it cannot compute and returns a plausible one is dangerous. A defect of exactly this kind occurred during development: a missing temperature channel was scored as “temperature is fine”, because a comparison against NaN evaluates to false, so every heat check silently passed for a pack nobody had measured. The design of BEACON takes the opposite stance: the system refuses to produce a number it cannot justify, and says why.

### 1.3 PROBLEM STATEMENT

The problem addressed in this project is how to build a software layer above a BMS that can measure battery health from the BMS’s own signals, namely voltage, current and time, without depending on a model trained on other cells, and that knows when its evidence is insufficient. Three sub-problems follow.

- **Measurement.** How can SOH, internal resistance and RUL be measured from voltage, current and time alone, including from partial discharges, and how accurately?
- **Honesty.** How can the system decide, without ground truth, when an estimate should be reported with a stated error band and when it should be refused?
- **Integration.** How can data from a CAN bus, a serial bench rig and research datasets all be scored through one shared pipeline and served to a dashboard, so that a difference in output is a difference in the telemetry and not in the code?

### 1.4 RESEARCH GAP

Data-driven SOH estimators are widespread, but how they are validated changes what they appear to achieve. Group-wise splitting by cell changes reported skill substantially [7], and a large gap between random and cell-wise splits has been reported on NASA data [8]. These studies stop at the cell. What remains insufficiently characterised is the next step: whether skill survives a change of *test protocol*, and what a BMS software layer can still measure reliably when it does not.

Curve-based methods, including differential voltage analysis [9] and incremental capacity analysis, including on-board use [11], read degradation from the shape of the voltage–charge curve. Online recursive estimation of cell parameters, including resistance, is established [5]. The gap this project addresses is the combination: estimators validated from raw telemetry rather than from laboratory summaries, with an online resistance correction taken from the load step, gates that refuse rather than extrapolate, and a measured criterion for when the evidence suffices, all evaluated on more than one dataset without re-tuning.

## 1.5 OBJECTIVES

The objectives of the project are:

1. To build an end-to-end telemetry system that ingests data from a CAN bus, a serial bench rig and research datasets, converges them on one schema, and scores them through one shared pipeline.
2. To estimate SOH from voltage, current and time alone, with an ohmic correction identified from the load step, and to validate it against laboratory capacity.
3. To report resistance as a second validated health quantity, and to estimate RUL from each cell's own fade trend, reported as a user meets it.
4. To measure rather than assert when the evidence suffices, so that every output is validated, banded by its measured error, or refused.
5. To validate externally, without re-tuning, on a second and third dataset, and to retain negative results.
6. To build and bring up a physical data-acquisition rig, and to serve all results through a REST API and a web dashboard.

## 1.6 KEY CONTRIBUTIONS

The main contributions of this work are as follows.

1. **A three-tier telemetry system** (CAN/serial/dataset ingestion, a shared scoring pipeline, and a FastAPI, Express and React service tier) in which every output is labelled measured, estimated or heuristic and carries its error band, or is refused with a reason.
2. **A field-realizable SOH estimator** validated from voltage, current and time alone, in which the ohmic correction uses a resistance identified from the voltage step at each load onset. Relative to no correction, it raises coverage from 12 to 17 of 22 cells and lowers median error from 4.3% to 1.7%.
3. **Resistance as a validated second health quantity:** the step resistance tracks the laboratory instrument on 19 of 20 cells.
4. **SOH from partial discharges** via a window learned from the cell's own early discharges, with a physical refusal rule for windows on the discharge knee whose threshold is not tuned.

5. **RUL accuracy reported as a user meets it**, indexed by the prediction, together with a measured correction (88% to 73%) caused by a reference capacity that used the future.
6. **Evidence sufficiency measured rather than asserted**: confidence follows the consistency of readings, which predicts error, and not the number of discharges, which does not.
7. **External validation without re-tuning** on a second dataset of a different chemistry, manufacturer, format and temperature, and a documented failure on a third.
8. **A working bench rig and software contract**: a serial wire protocol with checksums and refusal gates, firmware, and a physical bring-up on an ESP8266 board, with fault injection tests.
9. **Negative results retained**: fitted models across protocols and four routes to RUL from sparse complete discharges.

## 1.7 SCOPE AND ORGANISATION OF THE REPORT

Development and primary validation use single lithium cobalt oxide (LCO) cells at constant current (CALCE). External validation uses NMC/LCO blend pouch cells at 40 °C (Oxford), which also supply one real drive-cycle discharge. A third dataset (NASA) is used for the benchmark and for a cross-dataset test. The project makes no claim about packs, lithium iron phosphate, or temperature-dependent capacity correction. The bench rig validates data acquisition, not health estimation: no discharge has yet been recorded on it.

The remainder of the report is organised as follows. Chapter 2 reviews related work. Chapter 3 presents the system design and the estimation methods. Chapter 4 describes the implementation, the hardware rig and the deployment. Chapter 5 reports the results and discusses them. Chapter 6 concludes and lists future work.

## CHAPTER 2

### LITERATURE REVIEW

#### 2.1 STATE ESTIMATION IN THE BATTERY MANAGEMENT SYSTEM

A BMS protects the pack and estimates its state of charge from the terminal measurements it already holds. Reviews of monitoring methods for electric and hybrid vehicles [1] and of SOH estimation for real applications [2] distinguish direct measurement (capacity or resistance tests), model-based estimation and data-driven approaches. Ageing mechanisms and their effect on capacity and resistance are reviewed by Vetter *et al.* [3] and, for automotive applications, by Barré *et al.* [4]. Recursive estimation of cell parameters within the BMS, including internal resistance, is established [5], and monitoring of resistance is part of the methods reviewed by Waag *et al.* [1].

In this project, resistance serves two roles: it corrects the ohmic sag that would otherwise make a fixed voltage window slide along the discharge curve, and it is reported as a second health output, validated against the laboratory instrument (Section 5.4).

#### 2.2 DATA-DRIVEN SOH ESTIMATION AND HOW IT IS VALIDATED

Data-driven estimators trained on cycling features are widespread. Severson *et al.* [6] showed that features of the capacity–voltage curve, such as the variance of the difference between cycles, predict cycle life from early data. Such models are commonly assessed under random or leave-one-cell-out splits.

How they are validated changes what they appear to achieve. Group-wise splitting by cell reduces reported skill substantially [7], and a large gap between random and cell-wise splits has been reported on NASA data [8]. This project extends the hold-out from the cell to the test protocol, using leave-one-cohort-out validation with a bootstrap interval over folds, and finds that the interval on any single method is wider than the spread between methods (Section 5.1).

#### 2.3 DISCHARGE-CURVE ANALYSIS

Differential voltage analysis [9] and incremental capacity analysis, including on-board use [11], read degradation from the shape of the voltage–charge curve, and their features can be mapped to degradation modes [10, 12]. Estimators using partial charging curves have also been reported [13].

The window estimator used here belongs to this curve-based family. It differs in four ways: it uses no cross-cell fitting, it applies an online resistance correction from the

load step, it uses gates that refuse rather than extrapolate, and it is validated from raw telemetry rather than from laboratory summaries.

## 2.4 REMAINING USEFUL LIFE PROGNOSTICS

Remaining-useful-life methods and their uncertainty are reviewed by Hu *et al.* [14]. Accuracy is often summarised as a single error figure over all predictions. This project reports RUL error indexed by the prediction a user actually sees, for example “when the system predicts fewer than 25 cycles, how often did the cell last at least that long”, and records which routes to RUL failed (Sections 5.5 and 5.6).

## 2.5 PUBLIC DATASETS USED

Three public datasets are used, chosen because they differ in chemistry, format, temperature and protocol.

- **CALCE CS2 and CX2** [16]: prismatic LCO cells cycled at room temperature under several protocols, including multi-rate cycling and real partial cycling. Primary development and validation.
- **NASA PCoE** [15]: 18650 cells under nine documented protocols with ambient temperature from about 4 to 43 °C. Used for the benchmark and for a cross-dataset test.
- **Oxford Battery Degradation Dataset 1** [17, 18]: eight Kokam pouch cells aged at 40 °C with an Artemis urban drive-cycle discharge. Used for external validation.

## 2.6 TELEMETRY ACQUISITION IN VEHICLES AND ON THE BENCH

In a vehicle, BMS data is carried on a CAN bus, and a DBC file describes how signals are packed into frames. A research dataset arrives as files, and a bench rig typically streams text over a serial port. Each source has its own failure modes: a CAN decode can silently miss a signal, a serial link can drop or corrupt bytes, and a research file can store a quantity with a different sign or scale. The review above does not address how these three sources are made to agree, and this project treats it as a design question (Chapter 3): one schema, one scoring function, and explicit refusals when a capture’s shape is wrong.

## 2.7 POSITIONING OF THIS WORK

Table 2.1 summarises the three families of approach in this review by the properties that matter for a deployed software layer. It is a statement about the families as categories and not about any individual cited study.

Table 2.1: Approach Families and the Properties That Matter for a Deployed Layer

| Property                             | Fitted regression | Curve-feature methods | This work (BEACON)                   |
|--------------------------------------|-------------------|-----------------------|--------------------------------------|
| Needs a model trained on other cells | Yes               | Varies                | No                                   |
| Reference needed for the same cell   | No                | Varies                | First discharges only                |
| Typical hold-out unit in validation  | Cell              | Varies                | Whole protocol cohort, with interval |
| Refuses when evidence is missing     | Not by design     | Not by design         | Yes, with stated cause               |
| Works from partial discharges        | Varies            | Varies                | Yes, with a knee gate                |

## 2.8 SUMMARY

The literature establishes the measurements, the curve-based family of methods, online resistance estimation and the importance of validation splits. What it leaves open, and what this project addresses, is a software layer that is validated from raw telemetry across protocols and datasets, that fits nothing across cells, and that returns a refusal instead of a number when its evidence is insufficient.

## CHAPTER 3

### METHODOLOGY / SYSTEM DESIGN

This chapter explains *what* was designed, *why* it was designed in this way, and *how* the experiments were controlled. The practical realisation of the design (code organisation, wire protocol, hardware rig, services and deployment) is described in Chapter 4.

#### 3.1 PROBLEM AND DESIGN OBJECTIVE

The objective is a software layer above a BMS that converts voltage, current and time into health outputs an operator can act on, with two design constraints that shape everything below.

1. **Fit nothing across cells.** Chapter 5 shows that models fitted across cells do not retain skill under a change of test protocol. Both working estimators (SOH and RUL) are therefore expressed relative to the same cell's own early behaviour, so there is no training cohort to transfer from.
2. **Refuse what cannot be justified.** Every output is validated, banded by its measured error, or refused with a stated cause. A refusal is a correct outcome that the system knows, so it is returned as a value, rendered, and reflected in the process exit code.

##### 3.1.1 Research Variables

- **Independent variables:** the ohmic correction used (none, cyclor resistance, resistance from the load step), the voltage window (fixed or learned), the reference size, the RUL threshold, and the dataset (CALCE, Oxford, NASA).
- **Dependent variables:** per-cell mean absolute SOH error and its median over cells, coverage (cells measured out of cells scored), rank correlation of resistance, RUL error indexed by true and by predicted distance to end of life, and error stratified by the consistency of readings.
- **Controlled variables:** the reference definition (a quantity a BMS can hold), the primary window and tolerances, fixed before each study ran, and the scoring rule. Choices made after seeing a result are labelled exploratory.

## 3.2 OVERALL SYSTEM FRAMEWORK

BEACON ingests telemetry from three transports, converges them on one schema, and scores them through one shared function. The result, including every refusal, is served over an HTTP API to a React client (Fig. 3.1).

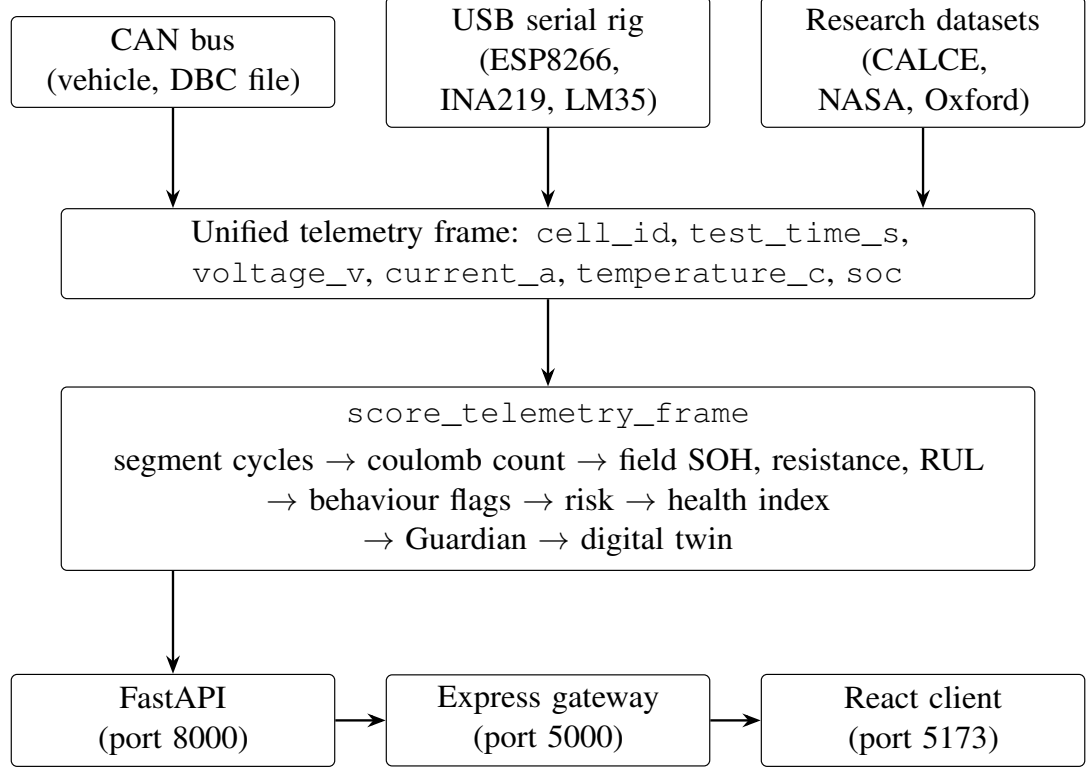


Figure 3.1: Overall system framework: three transports, one unified frame, one scoring function, and the service tier.

### 3.2.1 One Scoring Path, Extracted Rather Than Copied

The function `score_telemetry_frame` contains everything downstream of acquisition: segmentation, coulomb counting, features, risk, health, RUL, the Guardian explanation and the digital twin. It was extracted from the CAN pipeline when serial ingestion was added, instead of being copied. A second segmentation or feature extractor would mean that a disagreement between a CAN run and a serial run over the same battery could not be traced to either the data or the code. With the tail shared, a difference in output is a difference in the telemetry.

## 3.3 DATASETS

Four sources are used (Table 3.1). They were chosen because they differ in chemistry, format, temperature and protocol.

CALCE records span many files whose test time restarts per file. A monotonic time base is constructed by offsetting each restart, not by sorting timestamps, which

Table 3.1: Data Sources and Their Role

| Source           | Cells and conditions                                                                                       | Role                               | Ground truth used                    |
|------------------|------------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------|
| CALCE<br>CS2/CX2 | 22 prismatic LCO cells, room temperature, 7 protocols (constant-current, multi-rate, real partial cycling) | Development and primary validation | Cycler-measured discharge capacity   |
| NASA PCoE        | 18650 cells, 9 protocols, ambient 4–44 °C; 31 cells in the benchmark, 27 in the cross-dataset study        | Benchmark and third-dataset test   | NASA’s computed capacity             |
| Oxford Dataset 1 | 8 Kokam NMC/LCO pouch cells, 740 mAh, 40 °C, 519 characterisations                                         | External validation, unchanged     | 1C capacity at each characterisation |
| Bench rig        | 1 HONGLI 18650 cell, at rest, under 90 s per capture                                                       | Data acquisition only              | Frame integrity and timing           |

would interleave overlapping segments. The CALCE set includes real partial cycling (CS2 Types 5 and 6): thousands of discharges of about 25% depth in a fixed band, with a full capacity check roughly every 100 cycles. Type 6 cycles at the top of charge (about 4.07 to 3.78 V) and Type 5 near empty (about 3.69 to 2.70 V). The Oxford set additionally holds one real drive-cycle discharge of Cell 1 sampled at 1 Hz, used only for external validation.

### 3.4 FROM TELEMETRY TO DISCHARGES

The estimator receives voltage  $V(t)$ , current  $I(t)$  and time. The cycler’s capacity, resistance and cycle summaries are removed before scoring. Current is segmented into charge, discharge and rest phases using a dead band (0.02 A for CALCE, where rest is logged as exactly zero) and a minimum run length. The charge delivered on discharge  $n$  is the trapezoidal integral of  $|I| dt$ . Every discharge is retained: a partial discharge that crosses the window is a full measurement of the window.

### 3.5 VOLTAGE-WINDOW STATE OF HEALTH

For a window  $[V_{\text{low}}, V_{\text{high}}]$ , the window charge  $Q_w(n)$  is the charge delivered while terminal voltage falls from  $V_{\text{high}}$  to  $V_{\text{low}}$  on discharge  $n$ . It is read by interpolation and requires that the discharge span at least 95% of the window. With  $\mathcal{R}$  the cell’s first five measurable discharges,

$$\text{SOH}(n) = \frac{Q_w(n)}{\text{median}_{m \in \mathcal{R}} Q_w(m)}, \quad (3.1)$$

and the reported value is the median of the latest five accepted readings. The primary window, 3.90–3.60 V, was fixed before any validation run. The windows 4.05–3.55 and

3.85–3.55 V are reported as sensitivity analyses and not selected between.

Equation (3.1) assumes the discharge curve scales approximately uniformly with capacity. Loss of lithium inventory shifts the curve, which the ratio partly absorbs. Loss of active material and resistance growth change its shape and position, which it does not. This is the method’s principal approximation, and Section 5.7 measures how it drifts with age. Fig. 3.2 summarises the estimator.

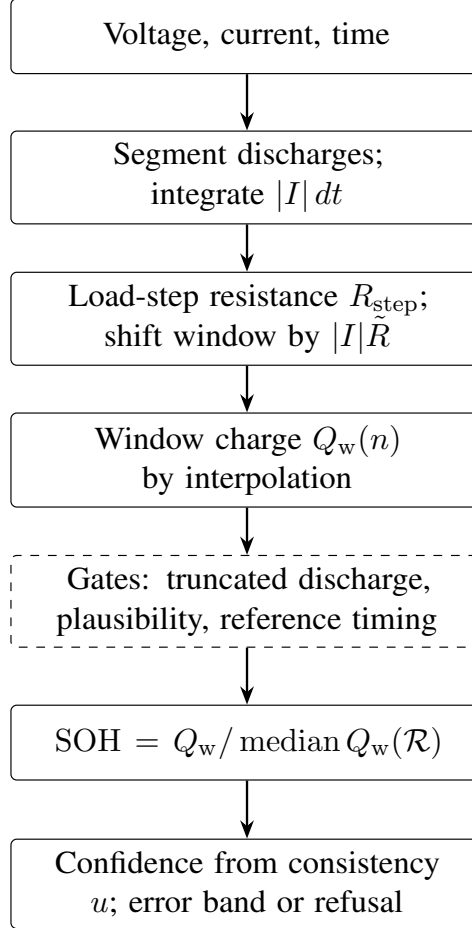


Figure 3.2: Field SOH estimator: from raw telemetry to a banded or refused output.

### 3.6 RESISTANCE FROM THE LOAD STEP AND OHMIC CORRECTION

Under load, terminal voltage lies below open-circuit voltage by approximately  $IR$ . As  $R$  grows with age, a fixed terminal-voltage window slides along the curve, and the slide reads as additional fade. At each discharge onset, between the last rest sample and the first loaded sample,

$$R_{\text{step}} = \frac{V_{\text{rest}} - V_{\text{load}}}{|I_{\text{load}} - I_{\text{rest}}|}, \quad (3.2)$$

accepted when the two samples are within 60 s and the current change is at least 0.05 A.  $R_{\text{step}}$  is an *apparent* resistance: the ohmic drop plus polarisation accrued by the first loaded sample. On cell CS2\_35 it is about 0.165  $\Omega$  against the cyclers’ about

0.099  $\Omega$ . Each discharge uses the trailing median  $\tilde{R}(n)$  of the last 11 step estimates, so no correction depends on later data, and its window is shifted to

$$[V_{\text{low}} - |I_n|\tilde{R}(n), V_{\text{high}} - |I_n|\tilde{R}(n)]. \quad (3.3)$$

Discharges preceding the first resistance estimate are left unmeasured rather than scored uncompensated.

### 3.7 GATES

Each gate needs no ground truth and can therefore run in deployment.

1. **Truncated discharge.** A discharge delivering less than 50% of the reference discharges' total charge is not scored. A cut-short cycle crosses the window with little charge and would read as severe fade.
2. **Physical plausibility.** A ratio below 0.50 is withheld, and a cell with more than 20% of readings below it is refused entirely. A ratio above 1.10 is withheld as not spanning the same part of the curve.
3. **Reference timing.** The reference must be complete within 20 equivalent full cycles of charge throughput. Throughput rather than a discharge count is used because storage-protocol records contain thousands of short pulses.

### 3.8 LEARNED WINDOWS FOR PARTIAL DISCHARGES

A car rarely discharges across a fixed window. When the fixed window cannot be read and the rated capacity  $C_{\text{rated}}$  is known, a window is learned from the cell's first ten discharges only. The band they all cover is trimmed by 15% at each end, and the flattest sub-window of up to 0.30 V within it is selected by its steepness

$$s = \frac{V_{\text{high}} - V_{\text{low}}}{Q_w / C_{\text{rated}}} \quad [\text{V per unit state of charge}]. \quad (3.4)$$

The window is refused if even the flattest choice has  $s > 2.0$ , because it then lies on the end-of-discharge knee. There, voltage reflects kinetics (resistance and solid-state diffusion) rather than stored charge.

### 3.9 REMAINING USEFUL LIFE FROM THE CELL'S OWN FADE

At cycle  $k$ , the cell's SOH history up to  $k$  is smoothed with an 11-point rolling median, a straight line is fitted to the full history, and its crossing of 0.90 gives the end-of-life estimate. Estimates are refused in three cases: with fewer than 30 points, for flat or rising trends, and when the estimate reaches more than twice the history length ahead. A cell already past the threshold is reported as zero. Linear, square-root and quadratic

forms, over full and trailing windows, were compared, and the full-history line was most accurate near end of life. No other cell's data enters, so there is no training cohort.

### 3.10 EVIDENCE SUFFICIENCY

Each output has its own requirements: capacity health needs voltage, current and five reference readings plus one; resistance health needs a clean load step; RUL needs 30 complete discharges; and heat exposure needs a temperature channel. Within capacity health, confidence follows consistency. Because a healthy estimate keeps changing as the cell ages, stability of the estimate is the wrong criterion. The question is whether readings agree once that trend is removed. With  $\sigma_{\hat{x}}(x_{1..n}) = 1.2533 \text{ MAD}(x)/\sqrt{n}$  the standard error of a median,

$$u_{\text{ref}} = \frac{\sigma_{\hat{x}}(Q_w(\mathcal{R}))}{\text{median } Q_w(\mathcal{R})}, \quad (3.5)$$

$$u_{\text{rec}} = \frac{1.2533 \text{ MAD}(\varepsilon)}{\sqrt{5}}, \quad (3.6)$$

$$u = \sqrt{u_{\text{ref}}^2 + u_{\text{rec}}^2}, \quad (3.7)$$

where  $\varepsilon$  are the residuals of the last ten readings about their own straight line. The tolerance  $u \leq 0.01$  (one SOH point) was fixed before the study of Section 5.7, chosen to lie below the validated 1.7% median error. Confidence is then assigned as *high* (consistent, fixed window and a beginning-of-life reference), *medium* (consistent, but a learned window or a start-of-log reference), or *low* (inconsistent, reported with the measured 22-point band).

### 3.11 BEHAVIOUR SCORING LAYER (HEURISTIC)

Alongside the measured estimators, BEACON carries a rule-based behaviour layer inherited from the project's first phase. It converts usage features into a risk score, an aging budget, a health index, a state label, a Guardian explanation and a digital-twin state. The features are average stress, average temperature, deep-discharge duration, fast-charge duration, aggressive-discharge count and average SOC.

Table 3.2: Risk Categories of the Rule-Based Layer

| Risk category | Risk score |
|---------------|------------|
| LOW           | 0–39       |
| MEDIUM        | 40–59      |
| HIGH          | 60–79      |
| CRITICAL      | 80–100     |

The aging budget starts at 100 and is reduced by penalties for stress, temperature, deep discharge, fast charging, aggressive discharge and SOC abuse, and the health index

Table 3.3: Health Index, Battery State and Digital-Twin State

| Health index | Battery state | Twin state       |
|--------------|---------------|------------------|
| 0–29         | HEALTHY       | NORMAL           |
| 30–59        | WARNING       | MODERATE_RISK    |
| 60–79        | DEGRADED      | HIGH_RISK        |
| 80–100       | CRITICAL      | FAILURE_IMMINENT |

is  $HI = 100 - \text{budget}$ , so a higher index means a worse battery (Tables 3.2 and 3.3). The digital twin is a one-to-one relabelling of the battery states, with a queryable API. It adds structure but no new predictive model.

This layer is *not* validated against measured degradation. The Spearman correlation between the risk score and measured fade on a separate set of 33 NASA cells was  $\rho = -0.27$  ( $p = 0.12$ ), so it is labelled a heuristic wherever it appears. Where measured SOH exists and the log starts at beginning of life, the measured SOH sets the battery state and records that it did, and the heuristic index is used only otherwise.

### 3.12 VALIDATION PROTOCOL

**Ground truth.** Ground truth is the cyclers’ measured discharge capacity, which never enters an estimate. SOH truth is capacity over the median of the cell’s first five cycles. For partial-cycling cells, whose checks are about 100 cycles apart, the partial-discharge study scores each cell against its first check, and the remaining-life study uses the capacity over the checks within the first ten cycles. Each is a reference a BMS can hold. A reference drawn from the cell’s whole record uses the future, and Section 5.5 quantifies the resulting inflation.

**Pre-declared configurations.** Primary windows, comparison arms, tolerances and scoring were fixed before each study ran. Choices made after seeing a result are labelled exploratory.

**Benchmark.** The benchmark uses leave-one-cell-out (LOBO, 31 folds) and leave-one-cohort-out (LOCO, 9 folds) validation.  $R^2$  is computed against the training-fold mean, with 95% bootstrap intervals resampling folds (2,000 resamples). Hyperparameters were fixed and not tuned per fold, except ElasticNet’s penalty, chosen by inner cross-validation within the training fold.

**Aggregation.** Errors are summarised per cell, then across cells, so long-lived cells do not dominate.

### 3.13 EVALUATION METRICS

- **SOH error:** per-cell mean absolute error in SOH (0.01 equals one percentage point), the median over cells, a 95% interval that resamples cells, and the worst cell.
- **Coverage:** cells measured out of cells scored. Refusals count against coverage and are reported with their cause.
- **Resistance:** Spearman correlation against the cyclers's resistance column over each cell's life, direction of growth, and growth ratio.
- **RUL:** the share of estimates within  $\pm 20$  cycles, the median error, and the median bias, by true distance to end of life and by predicted distance.
- **Sufficiency:** median and 90th-percentile error by consistency class, and the correlation between  $u$  and absolute error.
- **Benchmark:** LOBO and LOCO  $R^2$  against the training mean with bootstrap intervals, and LOCO mean absolute error.

### 3.14 EXPERIMENTAL CONTROLS AND REPRODUCIBILITY

Every reported number is regenerated by a named script from committed artifacts (Appendix A), and a test (`test_reported_numbers`) ties each figure quoted in the documentation to the artifact that produced it, so a re-run experiment with a stale paragraph fails the build. A control shuffles each cell's capacity in time to confirm that the benchmark harness does not manufacture skill. A continuous-integration job reproduces the headline benchmark end to end.

## CHAPTER 4

### IMPLEMENTATION AND DEPLOYMENT

#### 4.1 IMPLEMENTATION OVERVIEW

The previous chapter defined the design: one schema, one scoring function, two estimators that fit nothing across cells, and explicit refusals. This chapter describes how that design was realised. The implementation has three parts:

- **Part A – Telemetry and estimation software:** the packages that acquire, validate, segment and score telemetry (Sections 4.2 to 4.6).
- **Part B – Hardware bench rig:** the instrumentation design, firmware and physical bring-up that supply real telemetry to the software (Section 4.7).
- **Part C – Service tier and deployment:** the REST API, the gateway, the dashboard, testing, continuous integration and containers (Sections 4.8 to 4.11).

#### 4.2 DEVELOPMENT ENVIRONMENT AND TOOLS

The system is written in Python, Node (JavaScript), React and C++ (Arduino firmware), with about 23,000 lines of Python in the core package and 44 test modules. The components that matter for reproducibility are listed in Table 4.1.

#### 4.3 CODE ORGANISATION

The core package `src/bms` is divided into packages with one responsibility each (Table 4.2).

Outside the package, the repository holds the following.

- `mern/`: the Express gateway and React client.
- `firmware/beacon_rig/`: the Arduino sketch.
- `scripts/`: the study and demo scripts.
- `tests/`: the test modules.
- `docs/adr/`: fourteen architecture decision records, including those that record reversed decisions.

Table 4.1: Software Components Used

| Software                        | Role                                                                          |
|---------------------------------|-------------------------------------------------------------------------------|
| Python ( $\geq 3.10$ )          | Core language; CI tests 3.10 (the declared floor) and 3.12                    |
| pandas, NumPy, scikit-learn     | Telemetry frames, numerical work, benchmark estimators and bootstrap analysis |
| XGBoost, PyTorch (optional)     | Benchmark methods (gradient boosting, LSTM)                                   |
| FastAPI, uvicorn                | Scoring, telemetry ingest and fleet store as a REST service                   |
| cantools, python-can (optional) | DBC decoding and CAN capture; CI also runs the suite with them uninstalled    |
| pyserial (optional)             | Reading the physical rig’s serial port                                        |
| Express (Node)                  | Gateway between the client and the Python service                             |
| React, Vite                     | Dashboard client                                                              |
| Arduino C++ / arduino-cli       | Rig firmware, compiled for ESP32, ESP8266 and AVR                             |
| pytest, ruff, mypy              | Tests, linting and type checking                                              |
| Docker, Docker Compose          | Multi-stage image and three-service stack                                     |
| GitHub Actions                  | Eight CI jobs, including a reproduction of the headline study                 |

Table 4.2: Organisation of the Core Package

| Package                 | Responsibility                                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| telemetry               | CAN capture and replay, serial ingestion and protocol, cycle segmentation, and the shared scoring function |
| io                      | Dataset loaders (NASA, CALCE, Oxford), DBC examples, unified schema                                        |
| features, preprocessing | Behaviour features and data conditioning                                                                   |
| risk, health            | Rule-based risk score, aging budget and health index (heuristic layer)                                     |
| rul                     | Fade-extrapolation remaining useful life                                                                   |
| guardian, digital_twin  | Human-readable diagnosis and the state-machine twin                                                        |
| explain                 | Exact Shapley attribution over the rule scores                                                             |
| uncertainty             | Distribution-free uncertainty (conformal prediction) and coverage under shift                              |
| benchmarks              | Published methods run through the project’s own validation gate                                            |
| adaptive                | Governed model promotion across datasets and protocols                                                     |
| detect, physics         | Unsupervised usage segmentation; physics-grounded degradation models                                       |
| simulation              | Synthetic telemetry for tests and the no-hardware demo                                                     |
| api, dashboard          | FastAPI service and the static HTML dashboard                                                              |

## 4.4 TELEMETRY ACQUISITION: THREE TRANSPORTS, ONE FRAME

The three transports converge on one unified telemetry frame with the columns `cell_id`, `test_time_s`, `voltage_v`, `current_a`, `temperature_c` and `soc`.

- **CAN bus.** Frames are decoded against a DBC definition, and a coverage gate refuses a capture whose required signals are missing.
- **USB serial.** A line source feeds a parser that classifies each line, verifies its checksum and validates every field against a declared range (Section 4.5).
- **Research datasets.** Loaders read the NASA MATLAB files and the CALCE and Oxford records into the same frame.

### 4.4.1 The Seam Is Narrow

A serial source is a protocol with one method, `lines()`. A physical port, a recorded text file, a list in a test and the deterministic emulator are interchangeable, so the demo path and the hardware path differ in one constructor call. The emulator deliberately generates *wire-format text* and not record objects. Emitting objects would have been less code but would have proved nothing, because the parser, the checksum, the schema validation and the coverage gate would all be bypassed in exactly the path the tests exercise. The emulator even prints an ESP32 boot banner, so the sentinel filter runs on every test. Recording composes at the same seam: one decorator makes a live port, a replay or the emulator recordable, and the file on disk is exactly the bytes that produced the result.

## 4.5 SERIAL WIRE PROTOCOL

Each frame is a line of text with a sentinel, a record kind, a JSON body and an optional checksum (Fig. 4.1). The record kinds are HELLO (the rig declares its channels and rated capacity), D (data) and S (status).

```
BEACON1 D {"t":12.5,"v":3.91,"i":-1.48,"tc":27.4,"soc":82.1}*06
```

Figure 4.1: A data frame: sentinel BEACON1, kind D, JSON body, and the XOR checksum \*06, ending in a newline.

### 4.5.1 Framing by Sentinel

Framing is by sentinel and not by length. Every microcontroller prints boot banners and debugging text, so a length-prefixed parser would resynchronise after such noise only by luck. A sentinel parser discards anything not beginning with BEACON1 as counted noise and continues, which makes the protocol self-synchronising: it recovers at the next newline after any corruption, with a loss of at most one frame.

### 4.5.2 Error Detection

The checksum is an 8-bit XOR over the body (a longitudinal redundancy check). It detects all single-bit errors, but it cannot detect an even number of bit errors in the same bit position, any reordering of bytes, or corruption of the sentinel and kind fields, which lie outside the body. Under a random-corruption model, 0.39% of corrupted frames would pass. A CRC-8 occupies the same eight bits and detects all bursts up to eight bits and byte transpositions, so it is recorded as the protocol's clearest improvement. It was not adopted for the first bring-up because the XOR form is three lines of C and legible in a serial monitor.

### 4.5.3 Channel Utilisation

Table 4.3 gives the measured frame sizes and the derived link performance. At 1 Hz the link is over-provisioned by more than two orders of magnitude, so the verbose JSON codec is the default: it costs 39% more bandwidth than the compact codec and buys human-legible frames during bring-up.

Table 4.3: Frame Size and Link Performance of the JSON Codec

| Quantity                                  | Value        |
|-------------------------------------------|--------------|
| Data frame, mean size                     | 72.4 B       |
| Bits on the wire per frame                | 724          |
| Maximum sustainable rate at 115200 baud   | 159 frames/s |
| Utilisation at 1 Hz, 115200 baud          | 0.63%        |
| Utilisation at 1 Hz, 9600 baud (as built) | about 7.5%   |

## 4.6 REFUSAL GATES

A request through the serial path passes the following stages. The route validates the request, a line source yields lines (decode errors become counted rejections, not exceptions), the parser classifies each line and verifies its checksum, the serial pipeline applies the gates below and assembles the unified frame, and the shared scoring function runs. A decode-statistics record rides along the whole way so that a capture that silently dropped most of its lines cannot look like a clean one. Every gate returns a result carrying a refusal and not an exception (Table 4.4).

Two properties of this design are load-bearing. First, range checks *reject* rather than clamp: an out-of-range value is evidence that the reading is not what the schema says it is (a disconnected thermistor floats, an unpowered INA219 reads zero), and clamping would produce a value that looks measured and is not. Second, some errors are not expressible per record, so the capture-level checks sit one level above the record-level ones. Refusing is also not the same as withholding everything: segmentation and coulomb counting do not divide by capacity, so the capture is measured and reported

Table 4.4: Gates and What Each Prevents

| Gate                              | What it prevents                                                                                                                                               |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Missing required channel          | A comparison against <code>NaN</code> is false, so an absent temperature reads as “not hot” on every row. This defect shipped once and is why the gate exists. |
| Unimplemented schema id           | Identical field names meaning different things across a schema change.                                                                                         |
| SOC given as a 0–1 fraction       | A value that passes the range check but means the wrong thing. Caught per capture, because a fraction lies inside the 0–100 range.                             |
| Reversed current sign             | A rig reporting discharge as positive yields no discharge phases and no SOH while appearing to stream perfectly.                                               |
| Time running backwards            | A brown-out restarts <code>millis()</code> at zero, and coulomb counting would cancel the second session against the first. Sorting would hide it.             |
| Fewer than 50% of records parsed  | A wrong baud rate or mismatched firmware, as distinct from cable noise.                                                                                        |
| No rated capacity declared        | C-rate is current divided by rated capacity, and an assumed 2.0 Ah on a 3.4 Ah cell reads 1.7 C while drawing 1 C.                                             |
| No complete discharge cycle       | Capacity is comparable only across equal depths of discharge, so partial cycles are excluded and not scaled.                                                   |
| Nothing passed the promotion gate | The adaptive calibrator refuses to predict fade, and no transport routes around this.                                                                          |

and only the behaviour scoring is withheld. The capacity gate was moved to the scoring stage after the first implementation discarded valid cycle measurements.

## 4.7 HARDWARE BENCH RIG

### 4.7.1 Requirement and Design Calculations

The rig must measure one lithium-ion cell during charge and discharge and supply the five schema channels (terminal voltage, signed current, temperature, state of charge and elapsed time). State of charge is derived, not measured. The binding accuracy requirement is on capacity, with an acceptance criterion of within 5% of an independent reference.

The hardware design document works the instrumentation from datasheet figures; none of it has been measured on assembled hardware, and the calculations below are therefore design values. Current is measured high-side through a shunt read by an INA219, so that the terminal-voltage measurement is not shifted by the shunt drop. The shunt sizing is a trade between range, resolution and self-heating (Table 4.5).

Table 4.5: Shunt Sizing for the INA219 (320 mV Full Scale, 10  $\mu$ V LSB)

| $R_s$         | $I_{\max}$ | $I_{\text{LSB}}$ | Power at 2 A | Verdict                 |
|---------------|------------|------------------|--------------|-------------------------|
| 0.1 $\Omega$  | 3.2 A      | 100 $\mu$ A      | 0.40 W       | Selected for $\leq 3$ A |
| 0.05 $\Omega$ | 6.4 A      | 200 $\mu$ A      | 0.20 W       | 1C on a 3.4 Ah cell     |
| 0.01 $\Omega$ | 32 A       | 1.00 mA          | 0.04 W       | Wasteful here           |

Three consequences are recorded. Discharge must stay below 3.2 A with the 0.1  $\Omega$  shunt, because saturation is not a graceful failure: the reading clips, coulomb counting under-integrates and capacity reads low with nothing to indicate why. Self-heating shifts the shunt value, and a shunt reading high makes current read low, a systematic capacity error in one direction. The shunt drop is also subtracted from the voltage the INA219 reports on its load side, which is 200 mV (5% of a 4 V reading) at 2 A, so the bus voltage must be wired to the cell side of the shunt or corrected in firmware.

The error budget carries the component errors through to capacity. Combining the INA219 gain error (about 0.5%), the shunt tolerance (1%) and shunt self-heating (about 0.2%) in quadrature gives about 1.14% of reading, and because capacity is the integral of current, a gain error on current is a gain error on capacity. The expected capacity accuracy is about 1.2%, dominated by the shunt tolerance, against the 5% acceptance criterion. A 0.1% shunt would roughly halve it. Offset and timing contribute negligibly. These are datasheet-based design figures, and measurement against a reference meter has not been done.

### 4.7.2 As-Built Rig

The rig that was physically brought up differs from the design document in two components, and that difference is stated here and not hidden (Table 4.6, Fig. 4.2).

Table 4.6: Design Document Versus As-Built Rig

| Element             | Design document            | As built                            |
|---------------------|----------------------------|-------------------------------------|
| Microcontroller     | ESP32-WROOM-32 DevKit      | NodeMCU ESP8266 (CH340 USB bridge)  |
| Current and voltage | INA219 on I <sup>2</sup> C | INA219                              |
| Temperature         | DS18B20 (1-Wire)           | LM35                                |
| Cell                | Protected 18650            | HONGLI ICR-18650, single cell       |
| Serial rate         | 115200 baud                | 9600 baud (CH340 bridge limitation) |

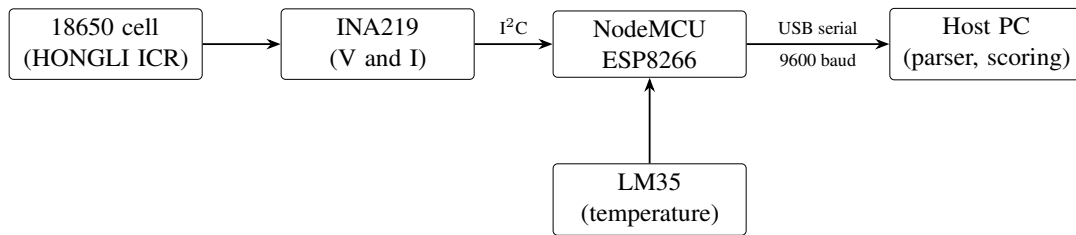


Figure 4.2: As-built bench rig: measurement chain from the cell to the host.

Protection must be in hardware in series with the cell, never in the monitoring firmware, because a firmware hang must not leave the cell unprotected. The monitoring electronics are powered from USB, not from the cell. The design power budget is about 48 mA typical against 500 mA available from USB, and the firmware keeps the radios disabled, not to save power but because transmit current spikes can brown out a weakly supplied board, which would restart the time base.

### 4.7.3 Firmware

The firmware is one board-agnostic Arduino sketch. The sensors sit behind four replaceable functions (`readVoltageV`, `readCurrentA`, `readTemperatureC` and `readSocPercent`), so a board with only some sensors can still run. At boot it probes each sensor, emits a status record naming what failed, and declares in its `HELLO` only the channels it can actually supply, together with the rated capacity. The host then scores the channels present and refuses the rest by name. The sketch compiles for three targets on every push (Table 4.7), and a test compares its declared field set, as a set, against the schema.

Table 4.7: Firmware Compilation Record (`arduino-cli`, all warnings enabled)

| Target               | Flash used                   | RAM used                 |
|----------------------|------------------------------|--------------------------|
| Arduino Uno (AVR)    | 8,990 B of 32,256 (27%)      | 460 B of 2,048 (22%)     |
| NodeMCU v2 (ESP8266) | 240,340 B of 1,048,576 (22%) | 28,372 B of 80,192 (35%) |
| ESP32 Dev Module     | 272,324 B of 1,310,720 (20%) | 22,140 B of 327,680 (6%) |

#### 4.7.4 Bring-Up Procedure

Bring-up was done in two stages that must not be combined. Stage A proves the transport with nothing at risk: no battery and no sensors. Stage B adds one cell and the sensors, replacing the four read functions one at a time. Doing both at once would make a wiring fault and a protocol fault indistinguishable with a lithium cell connected. Stage A passes when the accepted fraction is at least 0.99, a `HELLO` was seen, there are no time reversals, and the C-rate basis names the capacity the sketch declared, and when replaying the saved capture reproduces the live numbers.

A bare board can produce a full report: the stub firmware’s synthetic profile scores as complete cycles, so a rig with no sensors attached yields a health index and a remaining life that look like success. The records are well formed and in range, so the software cannot detect this, and the only defence is procedural: any hardware claim must state which of the four read functions were real. The captured file from a board with no sensors is kept in the repository as a labelled counter-example.

### 4.8 SERVICE TIER: REST API, GATEWAY AND DASHBOARD

The service tier has three components (Table 4.8). The gateway has exactly one job, translating HTTP calls from the client into calls to the Python service, and every route in it is a thin forward, so a wrong behaviour is fixed in the Python service and not in the gateway.

Table 4.8: Services

| Service             | Stack            | Responsibility                         |
|---------------------|------------------|----------------------------------------|
| api (port 8000)     | FastAPI, uvicorn | Scoring, telemetry ingest, fleet store |
| gateway (port 5000) | Express          | Client-facing aggregation              |
| client (port 5173)  | React, Vite      | Fleet table, detail, twin, replay      |

The principal endpoints are listed below, together with the telemetry routes for CAN replay, serial replay, coverage, the twin and the thermal view.

- GET /healthz: service liveness.
- POST /pipeline/simulate: run the pipeline on a simulated input.
- GET /batteries and GET /batteries/{id}: the battery list and one battery.
- GET /batteries/{id}/timeline: the per-battery timeline.

The /healthz endpoint is explicitly service liveness and not battery health: in a battery-monitoring system the naming collision is a real hazard and is called out at its definition. The container health check polls it.

The React client is a single-surface dashboard. It shows the fleet of batteries with health, risk and remaining life for each, a battery detail view, a digital-twin sheet with the twin state and its transitions, and a banner stating where the run's data came from. An earlier version had separate tabs for fleet, CAN replay, thermal and transfer views. They were removed because two parallel dashboards meant two provenance banners that could disagree about the same run. A Guardian explanation is shown next to the run it explains, and refusals are rendered as results and not as errors.

There are two deliberate absences. There is no database: the fleet store is in memory, and adding a database to the compose file would imply a persistence layer that does not exist, so the compose file says so. Persistence is the next substantial piece of work.

## 4.9 HEALTH REPORT CARD

The health report script renders the outputs as a plain-language card for one battery, in five parts: how healthy it is, how long it has left, what the usage is doing to it, what to do, and what the report cannot tell. Every figure carries its validation error band, which is taken from a file of measured bands and pinned by a test, and each output is labelled measured, estimate or heuristic. When ground truth exists, the card is followed by a check against the laboratory, which the card itself was not shown. Section 5.11 reports a worked example.

## 4.10 TESTING, CONTINUOUS INTEGRATION AND REPRODUCIBILITY

There are 44 test modules, and three kinds of test are not the usual kind.

- **Prose-to-evidence pinning.** A test ties every figure quoted in the documentation to the artifact that produced it, so re-running an experiment and forgetting to update a paragraph fails the build. It caught a stale figure that had already been published.
- **Cross-artifact structural tests.** The firmware's declared field set is compared as a set against the schema, and the documented schema table must equal the rendered

one. Substring assertions had already let a defect through: a renamed field left the schema id untouched, so the sketch kept announcing a version it no longer spoke.

- **Known-defect pins.** Some tests assert behaviour that is not desirable, with a comment saying so, for example that the risk score is too coarse to see a sevenfold change in average stress. Changing it would silently re-score every published figure, so it is pinned and a future change has to be deliberate.

Continuous integration runs eight jobs: lint and type checking, the test suite on Python 3.10 and 3.12, a job that uninstalls the optional CAN dependency and re-runs the suite, a job that reproduces the headline benchmark end to end, a firmware compile for three targets, the gateway tests, the client build, and the Docker build. The job without the optional dependency exists because “it is optional” is otherwise an untested claim. The Python-version matrix exists because testing only the newest version lets a declared floor rot silently.

#### 4.11 CONTAINERISATION AND DEPLOYMENT

The Docker image is multi-stage on a slim Python 3.12 base, with the compiler confined to the build stage, a non-root user, and a health check that polls service liveness. Docker Compose brings up the API, the gateway and the client, with the gateway held until the API reports healthy, and a separate study service writes the result CSVs back to the host. The system runs on a fresh clone with no hardware, no database, no API keys and no dataset download:

```
python main.py                    # end-to-end run
python scripts/run_serial_demo.py # emulated rig
python -m pytest tests/ -q        # the test suite
docker compose up                 # API, gateway, client
```

#### 4.12 CHAPTER SUMMARY

This chapter described the realisation of the design: a shared scoring path fed by three transports through a narrow seam, a self-synchronising wire protocol with refusal gates at both record and capture level, a physical rig brought up in two stages, a three-service stack served from containers, and a test and CI strategy aimed at keeping the documentation and the code honest. What this implementation *achieved* is reported in the next chapter.

## CHAPTER 5

### RESULTS AND DISCUSSION

This chapter reports what the system measured and interprets each result where it appears. Every number is traced to a committed artifact (Appendix A). Errors are mean absolute errors in SOH (0.01 equals one percentage point) per cell, summarised as a median over cells with a 95% bootstrap interval that resamples cells, unless stated otherwise. Results that failed are reported alongside those that worked.

#### 5.1 FITTED MODELS DO NOT SURVIVE A CHANGE OF PROTOCOL

On the NASA benchmark (31 cells, nine cohorts; the isotonic signal ceiling for the target SOH is 0.569), every one of twelve published methods lost skill from leave-one-cell-out (LOBO) to leave-one-cohort-out (LOCO) validation (Table 5.1). LOBO rank carried almost no information about LOCO rank (Spearman  $\rho = +0.084$ ,  $p = 0.795$ ,  $n = 12$ ). Across fourteen scored methods, the spread of LOCO point estimates (2.06 in  $R^2$ ) was smaller than the width of the best method’s own interval (3.66).

Table 5.1: Benchmark on 31 NASA Cells in Nine Cohorts (Six of the Methods Shown); LOCO Intervals Are 95% Bootstrap Over Folds

| Method                  | LOBO<br>$R^2$ | LOCO<br>$R^2$ | LOCO 95% CI   | LOCO<br>MAE |
|-------------------------|---------------|---------------|---------------|-------------|
| XGBoost                 | 0.732         | 0.459         | [−2.80, 0.86] | 7.62%       |
| Age-linear baseline     | 0.483         | 0.406         | [−0.43, 0.51] | 10.71%      |
| ElasticNet              | 0.609         | 0.355         | [−1.65, 0.76] | 10.31%      |
| GPR (Matérn)            | 0.725         | 0.207         | [−3.72, 0.43] | 17.06%      |
| Random forest           | 0.773         | 0.150         | [−1.09, 0.90] | 10.67%      |
| LSTM (10-cycle windows) | 0.593         | −0.185        | [−2.09, 0.66] | 17.36%      |

An ablation isolates where the skill comes from (Table 5.2). XGBoost on cycle number alone scored a LOCO  $R^2$  of  $-0.27$ , and on behaviour features alone  $-0.29$ . Only their combination reached 0.459, while a straight line on cycle number reached 0.406. Shuffling each cell’s capacity in time placed every LOCO interval at or below zero, so the harness does not manufacture skill. Usage features therefore added about 0.05 in  $R^2$  over counting cycles, within the intervals.

Temperature was the only usage factor with a consistent, correctly signed within-protocol relationship to fade (seven of seven cells), and its magnitude did not transfer between protocols. The rule-based risk score did not track measured fade either ( $\rho = -0.27$ ,  $p = 0.12$  on 33 cells), so it is labelled a heuristic.

Table 5.2: Feature Ablation and Shuffle Control (NASA SOH, LOCO)

| Run                        | Method     | LOCO $R^2$ | 95% CI          |
|----------------------------|------------|------------|-----------------|
| All features (published)   | XGBoost    | 0.459      | [−2.80, 0.86]   |
| All features (published)   | Age-linear | 0.406      | [−0.43, 0.51]   |
| Cycle number only          | XGBoost    | −0.266     | [−1.15, 0.82]   |
| Behaviour features only    | XGBoost    | −0.293     | [−0.72, 0.80]   |
| Shuffled in time (control) | Age-linear | −0.039     | [−0.35, −0.004] |
| Shuffled in time (control) | XGBoost    | −0.436     | [−1.44, −0.008] |

**Interpretation.** No fitted method retains reliable skill under a change of protocol, and no method can be distinguished from another. This is the result that motivates the rest of the project: the estimators that ship fit nothing across cells.

## 5.2 FIELD SOH FROM VOLTAGE, CURRENT AND TIME

Table 5.3 compares three ohmic treatments at the primary window on 22 CALCE cells. The field correction matched the laboratory correction within its interval and measured five more cells than no correction. Sensitivity windows gave 1.2% (4.05–3.55 V, 16 cells) and 1.9% (3.85–3.55 V, 17 cells).

Table 5.3: Field SOH at the 3.90–3.60 V Window; Median Per-Cell Absolute Error Against Cyclor Capacity, 95% Bootstrap Interval Over Cells

| Ohmic correction                    | Cells        | Median      | 95% CI          | Worst        |
|-------------------------------------|--------------|-------------|-----------------|--------------|
| None                                | 12/22        | 4.3%        | 2.5–5.4%        | 6.4%         |
| Cyclor resistance (laboratory only) | 12/20        | 1.4%        | 1.2–2.5%        | 5.6%         |
| <b>Step resistance (field)</b>      | <b>17/21</b> | <b>1.7%</b> | <b>1.4–4.3%</b> | <b>10.3%</b> |

The denominators differ because a correction can only be scored on cells for which it exists: one cell never rests before discharging, so no step resistance exists for it, which gives 17 of 21 cells for the field arm and 17 of 22 once that cell is counted as refused. The three worst cells (6.4–10.3%) are all Type 3, whose discharge switches rate within a cycle. Every refusal carries a stated cause: four storage and partial-protocol cells delivered 80–155 equivalent full cycles before any constant-current discharge crossed the window, so the reference gate refused them, and one cell never rests before discharging. Fig. 5.1 tracks the estimate against every laboratory capacity test, and Fig. 5.2 shows every measured cell.

**Interpretation.** Identifying resistance from the load step took the median error from 4.3% to 1.7% and the coverage from 12 to 17 cells, without any quantity the cyclor computed. Two earlier figures were corrected downward by this work: an earlier 3.3% SOH figure and an 88% RUL figure had been scored against a reference capacity taken from each cell’s whole life, which uses the future and which no BMS can hold.

State of health from voltage, current and time only, against lab capacity tests

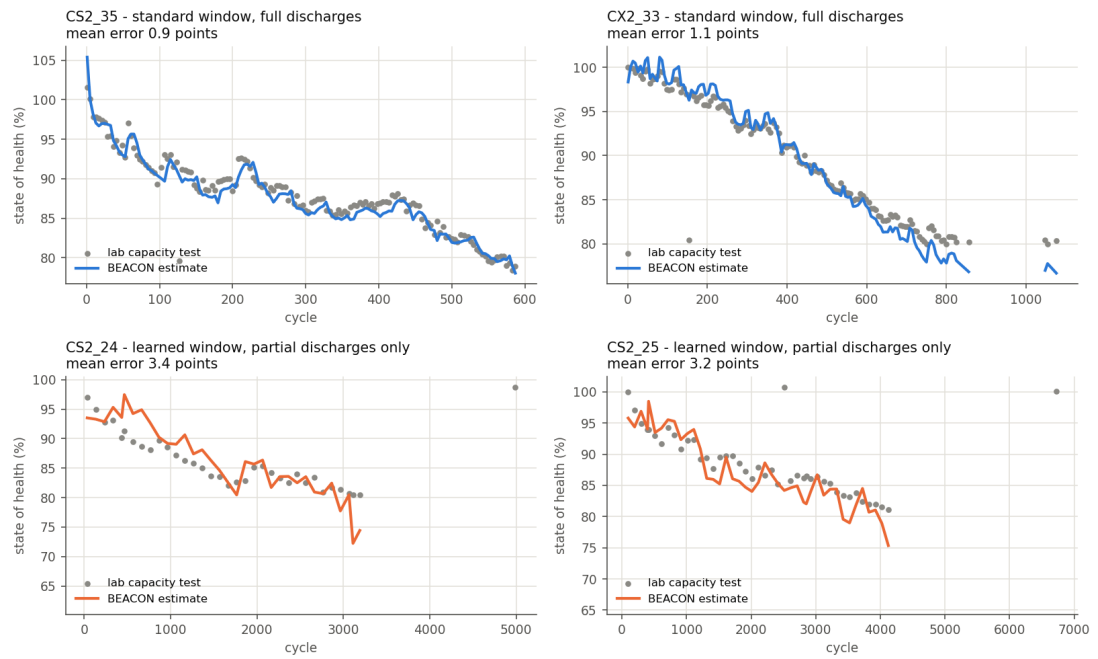


Figure 5.1: SOH from voltage, current and time against every laboratory capacity test. Top: two constant-current cells, fixed window (mean error 0.9 and 1.1 points). Bottom: two cells of real partial cycling, learned window, estimate from partial discharges only (3.4 and 3.2 points).

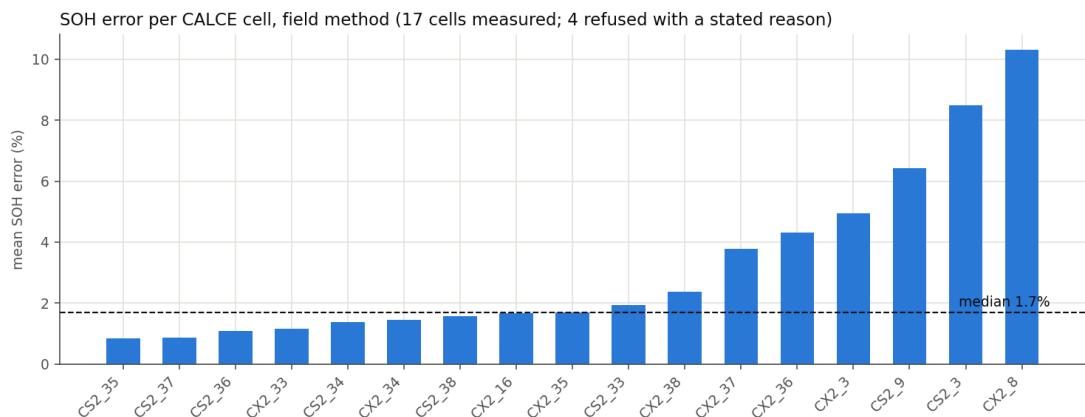


Figure 5.2: Mean absolute SOH error for every measured CALCE cell, field method; the dashed line is the median (1.7%).

### 5.3 PARTIAL DISCHARGES

On the four cells of real partial cycling (Table 5.4), estimates use only the partial discharges preceding each laboratory check. Before the knee gate existed, the two near-empty cells were reported at 6.8% and 12.8% error while appearing confident. Learned windows on the 18 full-discharge cells have steepness 0.44–0.88, so any threshold between about 0.9 and 8 gives the same verdict on every cell and the chosen 2.0 is not tuned. As a control, the learned mode on those 18 cells measured 16 at a median of 1.95%, beside 1.7% for the fixed window, so it is a fallback and not a replacement.

Table 5.4: SOH from Real Partial Cycling (CALCE CS2 Types 5 and 6)

| Cell   | Band          | Learned window | Steepness | Result         |
|--------|---------------|----------------|-----------|----------------|
| CS2_24 | top of charge | 4.00–3.82 V    | 0.83      | 3.4% error     |
| CS2_25 | top of charge | 4.03–3.83 V    | 0.82      | 3.2% error     |
| CS2_5  | near empty    | –              | 10.4      | refused (knee) |
| CS2_6  | near empty    | –              | 8.2       | refused (knee) |

**Interpretation.** The refusal is physics and not a tuned cut. On the plateau, window charge tracks capacity, while on the knee voltage is set by resistance and diffusion, so it measures how hard the cell is working instead. The scope is two cells per band, which shows the mechanism and is too few to quote a population error.

### 5.4 RESISTANCE TRACKS THE INSTRUMENT

Against the cyclers’s internal-resistance column over each cell’s life (20 cells carrying both values), the step resistance achieved a median Spearman correlation of 0.86, growth in the same direction on 19 of 20 cells, and a median growth of 1.26 times against the cyclers’s 1.29 times. This includes the partial-cycling cells (0.90 and 0.98), and the exception is a Type 3 cell. The levels differ, since the step value includes early polarisation, but the trends agree, which is what a power-fade indicator requires.

### 5.5 REMAINING USEFUL LIFE

At threshold 0.90, referenced to each cell’s first cycles (17 cells), Table 5.5 reports accuracy by the true distance to end of life and by the prediction a user sees. Near end of life the estimate is a safe lower bound: it warns early and almost never late. Errors are predominantly early because these cells fade quickly at first and then more slowly, so a full-history line is steeper than the remaining fade.

Scored against a reference capacity taken as the 95th percentile of the cell’s whole record, the same estimator was within  $\pm 20$  cycles 88% of the time near end of life. That reference uses the future, which no BMS can hold. Against the cell’s own early capacity the figure is 73%, and that is the figure reported. Fig. 5.3 summarises the result.

Table 5.5: RUL to 90% Capacity. Left: By True Cycles to End of Life. Right: By Predicted Cycles, With the 90% Range of (True – Predicted)

| True    | Within $\pm 20$ | Median err. | Predicted | Lasted $\geq$ pred. | 90% range   |
|---------|-----------------|-------------|-----------|---------------------|-------------|
| 0–25    | 73%             | 9.6         | 0–25      | 95%                 | +1 to +82   |
| 25–50   | 44%             | 22.8        | 25–50     | 88%                 | –14 to +129 |
| 50–100  | 15%             | 42.7        | 50–100    | 85%                 | –40 to +141 |
| 200–400 | 0%              | 144         | –         | –                   | –           |

Remaining life to 90%: 73% within  $\pm 20$  cycles in the last 25 cycles (median miss 10 cycles)

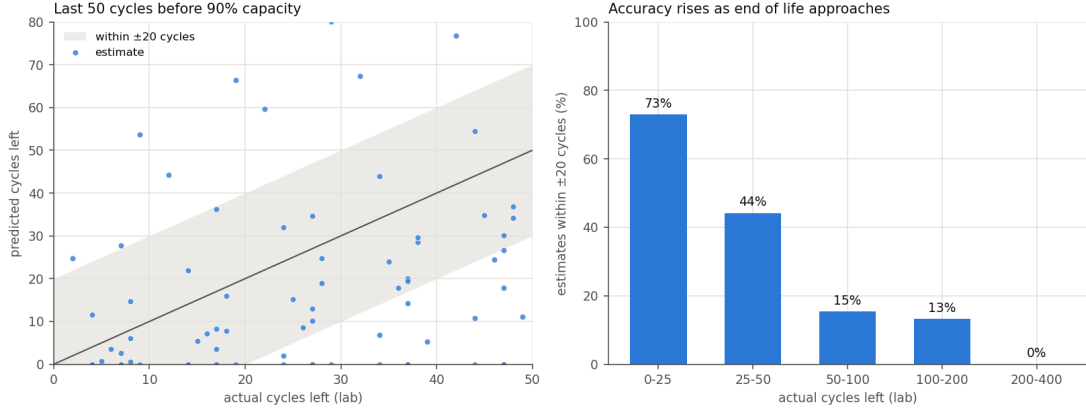


Figure 5.3: RUL to 90% capacity. Left: predicted against actual cycles left in the last 50 cycles. Right: share of estimates within  $\pm 20$  cycles by actual distance to end of life.

**Interpretation.** The “within  $\pm 20$  cycles” claim is true near end of life and false far from it (0% at 200 or more cycles out), so it is reported with its condition and not bare. The conservative direction is a property of the 90% threshold on these cells and is not a general guarantee. At an 85% threshold the median bias near end of life turns positive (+6.7 cycles within 25 cycles of end of life), so the estimate runs late there and only 62.8% of estimates fall within  $\pm 20$  cycles. The conventional 80% end-of-life threshold is not validated on CALCE, because its full-discharge trajectories end near 0.81 and only one cell of 22 crosses 0.80.

## 5.6 RUL WHEN COMPLETE DISCHARGES ARE RARE

A driver who seldom fully discharges produces few complete discharges (CS2\_24 has 18 in 1,617). Four methods were evaluated on the same sparse evidence (Table 5.6). Sparsity is real on the partial-cycling cells and emulated as one complete discharge in 50 on the constant-current cells.

The failure of the field-trajectory method has a clean cause. Near 0.90, field SOH reads a median 1.1 points below capacity (measured in a first pass of this study, not kept as a separate artifact). Because fade near the threshold is slow, that small bias moves the 0.90 crossing a median 56 cycles early. An SOH error that is acceptable as a health

Table 5.6: RUL from Sparse Complete Discharges; None Is Accurate Enough to Report

| Method                                                 | Outcome                                                     |
|--------------------------------------------------------|-------------------------------------------------------------|
| Sparse capacity, standard minimum                      | Refuses every point (never reaches 30)                      |
| Field SOH trajectory                                   | Answers 62%; median miss 79 cycles when predicting under 25 |
| Field SOH scaled by the cell's own complete discharges | Over-corrects; errors turn late near end of life            |
| Sparse capacity, minimum lowered to 5 (post hoc)       | 0% within $\pm 20$ cycles; misses late                      |

figure thus becomes large as a time to threshold. None of the four is reported, and the system refuses RUL for partial-only records and states that this was tested.

### 5.7 WHEN THE EVIDENCE SUFFICES

Three questions were fixed before the study and scored at every laboratory check on 19 cells. First, *count does not predict accuracy*: median error was 0.75% at 6–10 usable readings and 1.97% beyond 60, because error grows with age as the approximation in Eq. (3.1) drifts. A rule of high confidence after a fixed number of discharges therefore has no support. Second, *consistency does* (Table 5.7). The pooled Spearman correlation between  $u$  and absolute error was 0.48. Within cells it was weaker (median 0.14, positive in 14 of 19), so  $u$  chiefly separates unreliable cells and periods and does not rank readings within a good one.

Table 5.7: Error by the Consistency Criterion of Eq. (3.7)

| Readings                   | Points | Cells | Median error | 90th percentile |
|----------------------------|--------|-------|--------------|-----------------|
| $u \leq 0.01$ (consistent) | 8,993  | 16    | 1.3%         | 4.4%            |
| $u > 0.01$ (inconsistent)  | 2,168  | 13    | 5.1%         | 21.9%           |

Third, *reference size*: in the study's own scoring, larger references reduced error (one discharge 3.6%, five 2.0%, ten 1.3%), but on the field-study metric ten versus five changed the median only from 1.681% to 1.665%, so the estimator retains five.

### 5.8 EXTERNAL VALIDATION ON A SECOND DATASET

The estimator was applied to the eight Oxford cells without re-tuning. Arms, reference and scoring were fixed before the run. The reference is each cell's first characterisation, and truth is 1C capacity relative to it. Because each characterisation block begins under load, no rest-to-load step exists, so the 1C window is uncorrected.

All eight cells were measured (Table 5.8). The near-equilibrium C/18 window was not more accurate than the 1C window, so on this blend cathode the residual error stems

Table 5.8: External Validation on Eight Oxford Cells: Median and Worst Per-Cell Mean Absolute SOH Error

| Window                             | Cells | Median | Worst |
|------------------------------------|-------|--------|-------|
| Fixed 3.90–3.60 V, 1C discharge    | 8/8   | 3.0%   | 4.0%  |
| Fixed 3.90–3.60 V, C/18 pseudo-OCV | 8/8   | 3.2%   | 3.4%  |
| Learned window, 1C discharge       | 8/8   | 2.3%   | 3.6%  |

from the curve changing shape with age and not from ohmic sag, consistent with the stated approximation of Eq. (3.1).

**A real drive-cycle discharge.** The drive-cycle discharge of Cell 1 has current between  $-5.0$  and  $+1.6$  A. Resistance from its own 1,575 current steps of at least 0.2 A gave a median of 21 m $\Omega$  (interquartile 19–24). The discharge is itself partial (it stops at about 65% depth), so a coverage window (4.08–3.78 V) was chosen from the corrected curve’s span alone, before any comparison was computed. In it the drive-cycle reading, corrected per sample and fitted monotonically, was 0.2625 Ah, against 0.2661 Ah from the same cell’s 1C constant-current discharge, a difference of 1.4%. Both lay about 13% below the C/18 reading in that window, so the rate effect cancels in a ratio of like with like but would bias one that mixed rates. This is a single discharge and demonstrates the mechanism, not a population error.

**Remaining life at 80%.** The Oxford trajectories cross the 80% convention, so the shipped RUL estimator was scored there unchanged. When it predicted fewer than 500 cycles to 80%, the cell lasted at least that long in 97% of cases (36 estimates, 5 cells). Precision is low: the median miss is 385 cycles against characterisations 100 cycles apart. The guarantee also weakens with distance, since predictions of 500 to 1,000 cycles were met or exceeded only 80% of the time (25 estimates, 5 cells).

## 5.9 A THIRD DATASET AND KNOWING WHEN TO REFUSE

The NASA files (18650 cells, ambient 4–44 °C) were read from their original MATLAB form and scored the same way, unchanged. The estimator was poor on NASA overall, with a median per-cell error of 9.2% over 27 cells, and it depended on temperature: 1.7% in the 43 °C cohort and 12–14% at 4 and 22 °C. The step resistance captures only the ohmic part of the overpotential, and at low temperature the charge-transfer and diffusion parts, which grow with age, move the curve through the window.

The consistency uncertainty, with its tolerance fixed on CALCE, was applied unchanged. On NASA, estimates it labelled HIGH or MEDIUM had a median per-cell error of 4.5%, while those labelled LOW had 13.2% (within three SOH points: 49% versus 12%). The failure on unseen conditions was thus largely signalled by the estimator

itself.

For contrast, a gradient-boosted model on three window features, the setting where fitting looks best, reached 0.7% under leave-one-cell-out within Oxford and 1.2% within CALCE, but 6.7% when moved from Oxford to NASA and 5.5% from CALCE to Oxford. Validation inside one dataset overstated transferable accuracy four- to tenfold.

## 5.10 SYSTEM AND HARDWARE RESULTS

### 5.10.1 Bench Rig Captures

Three captures are committed (Table 5.9). The first is a counter-example: a board with no sensors attached, whose synthetic profile still scores, and which is kept and labelled as such.

Table 5.9: Bench Rig Captures

| Capture              | Declared channels                         | Records      | Measured                                                      |
|----------------------|-------------------------------------------|--------------|---------------------------------------------------------------|
| Stage A (no sensors) | all, from stubs                           | 151          | Synthetic, not a hardware result                              |
| Stage B voltage      | $t, v, i$ , SOC (INA219 up, LM35 refused) | 83 over 82 s | 3.9871 V (sd 1.9 mV); current at the $\pm 0.4$ mA noise floor |
| Temperature demo     | $t, t_c$ (LM35 up, INA219 refused)        | 85 over 84 s | 32.11 °C (sd 0.086 °C)                                        |

In both real captures the cadence held at 1.000 s with no gaps and a monotonic time base. Every measured voltage is an exact multiple of the INA219’s 4 mV bus LSB, which is what a genuine read from that part looks like and what a fabricated one would not. The sensor-complement declaration worked as designed: the firmware named the sensor that failed at boot, declared only the channels it could supply, and the host scored those and refused the rest by name. The first real board also exposed a baud-rate fault (the CH340 bridge accepts only 9600 and 19200 baud), which the refusal wording named as a cause and which led to the fix.

### 5.10.2 Fault Injection

Faults injected into a real capture behaved as designed: a corrupted byte failed its checksum, a NaN field was rejected, a reversed timestamp refused the capture, and a missing sensor was refused by name. The injection also exposed one defect, since corrected: a 999 V reading passed the field range, which is wide so that packs are admissible. Records outside 0–5 V are now rejected when the source declares a single cell.

### 5.10.3 What the Hardware Result Does and Does Not Show

The captures demonstrate that the transport, the protocol, the gate and the refusal path work against real silicon. They are one cell, at rest, for under 90 s, with no load step and

no charge or discharge cycle. They are *not* a validation of the health, RUL or risk models, which were fitted on cycled research cells and cannot be confirmed or refuted by a cell that was never cycled. Sensor accuracy against a reference meter was not measured, and no discharge has yet been recorded on the rig.

### 5.11 A WORKED EXAMPLE: THE REPORT CARD

Fig. 5.4 abridges the report card for CALCE cell CS2\_35 as it would have read at cycle 90. The pipeline saw voltage, current and time only.

```

1. HOW HEALTHY IT IS
   91.0% state of health [MEASURED] State: WARNING
2. HOW LONG IT HAS LEFT
   About 5 cycles until it falls to 90% of its early
   capacity. [ESTIMATE]
3. WHAT YOUR USAGE IS DOING TO IT
   No temperature was measured, so heat exposure cannot
   be assessed.
4. WHAT TO DO
   Capacity fade is measurable. Keep monitoring.
5. WHAT THIS REPORT CANNOT TELL YOU
   - Scoring skipped: required channels not supplied.

CHECK AGAINST THE LAB (the card was not shown this)
  Lab capacity at cycle 90: 91.4%; card said 91.0%
  Cell actually crossed 90% at cycle 146: 56 cycles
  after cycle 90; card said 5

```

Figure 5.4: Abridged health report card for CS2\_35 at cycle 90, with the laboratory check.

The health figure is within half a point. The remaining-life figure is 51 cycles early, which is the conservative bias the RUL study measured at that distance (median –32 cycles at 50–100 cycles out), and it is the reason the card prints its own accuracy beside the number and not the number alone. Only temperature is offered as a usage finding, because it is the one behaviour-to-ageing link that survived testing, in direction only.

### 5.12 DISCUSSION

**Conditioning moved accuracy; model choice did not.** Across fourteen scored methods, LOCO differences were smaller than the uncertainty on any one of them. Three conditioning decisions, by contrast, changed error by multiples. Refusing truncated discharges took the worst cell from 52% to about 10%. Identifying resistance from the load step took the median from 4.3% to 1.7% and coverage from 12 to 17 cells. Refusing knee windows converted 6.8–12.8% errors into refusals. For a BMS software layer, the leverage lies in what is measured and how it is conditioned, not in the regressor that follows. The first of these was a misdiagnosis corrected during the project: the 52%

worst case had been attributed to loss of active material, when it was actually truncated discharges being read as full ones.

**Reference definitions change conclusions.** Two results moved when the reference was made one a BMS can hold: RUL accuracy fell from 88% to 73%, and a partial-discharge reference formed from a late discharge produced a large error until the timing gate existed. Validation should use the reference the deployed system will have.

**Threshold crossings amplify small biases.** A 1.1-point SOH bias, unremarkable as a health figure, shifted predicted end of life by about 56 cycles. Any RUL built on an estimated rather than measured SOH inherits this, and should be evaluated as a time to threshold and not through SOH error.

**Sufficiency is measurable.** Treating “enough data” as a count is intuitive and here wrong, because the estimator’s error is dominated by age-related drift and not sampling. The consistency of readings is observable without ground truth and separates 1.3% from 5.1% median error, which gives a BMS layer a principled way to report “not yet” for a battery it has never seen.

### **Limitations.**

- *Temperature.* On NASA cells at 4 and 22 °C the estimator erred by 12–14%. The low-confidence label marked most of these readings, but a temperature-aware correction is future work.
- *Two cathode families, single cells, one real drive cycle.* Packs add cell-to-cell imbalance, thermal gradients and balancing currents, and the flat plateau of lithium iron phosphate would challenge the window method directly.
- *Two cells per partial-cycling band.*
- *End of life:* validated at 90% on CALCE, and at 80% on Oxford only as a conservative bound with low precision.
- *Apparent resistance* depends on the logger’s sample interval and is comparable within a record, not across loggers.
- *No hardware discharge* has yet been recorded on the bench rig, and the heuristic risk layer is not validated.

## **5.13 VALIDATION SUMMARY**

Table 5.10 lists the outputs by status. Claims that failed or were never tested are listed, because that is what makes the others believable.

Table 5.10: Status of the Main Outputs

| Output                               | Result                                  | Status                        |
|--------------------------------------|-----------------------------------------|-------------------------------|
| Capacity health (full discharges)    | 1.7% median, 17 of 22 cells             | Validated                     |
| Capacity health (partial discharges) | 3.2–3.4%, 2 cells                       | Validated, 2 cells            |
| Resistance health                    | Same direction on 19 of 20 cells        | Validated                     |
| RUL near end of life                 | 73% within $\pm 20$ cycles              | Validated, 90% threshold      |
| RUL far from end of life             | 0% at 200–400 cycles                    | Not valid (shown with band)   |
| RUL from sparse complete discharges  | Four methods, none accurate             | Refused                       |
| Second dataset (Oxford)              | 3.0% median, 8 of 8 cells               | Validated                     |
| Third dataset (NASA)                 | 9.2% overall, 1.7% at 43 °C             | Fails outside warm conditions |
| Fitted models across protocols       | Best LOCO $R^2$ 0.459, CI [−2.80, 0.86] | Not generalising              |
| Heuristic risk score                 | $\rho = -0.27$ , $p = 0.12$             | Labelled heuristic            |
| Telemetry on hardware                | 83/83 frames, 1.000 s cadence           | Validated (acquisition only)  |
| Health from a real rig discharge     | No discharge recorded                   | Not tested                    |
| Packs, LFP, large-scale drive cycles | No data                                 | Not tested                    |

## CHAPTER 6

### CONCLUSION AND FUTURE WORK

This project asked whether a software layer above a battery management system can turn voltage, current and time into a trustworthy battery health report, and when it should decline to answer. It built BEACON, a three-tier system that ingests telemetry from a CAN bus, a USB serial rig or research datasets, converges all three on one schema, and scores them through one shared pipeline that is served to a React dashboard.

The research answer is that such a layer can measure cell health without any model trained on other cells. On 22 CALCE cells it delivered capacity health at 1.7% median error (95% CI 1.4–4.3%) on 17 cells, with every one of the five refusals explained, and a resistance trend that tracked the laboratory instrument in the same direction on 19 of 20 cells. Partial-discharge capacity health reached 3.2–3.4% on two cells of real top-of-charge cycling, with a physical rule that refuses windows on the discharge knee. Unchanged, the estimator measured all eight cells of an independent NMC/LCO dataset at 40 °C with a 3.0% median error, and a real drive-cycle discharge read within 1.4% of a constant-current one. Remaining useful life extrapolated from each cell’s own fade was within  $\pm 20$  cycles 73% of the time in the last 25 cycles before 90% capacity, and at that threshold behaved as a safe lower bound.

Equally important are the results that failed. Twelve published fitted methods all lost skill when the held-out unit changed from a cell to a protocol cohort, and no ranking of methods was supportable, which is why the shipped estimators fit nothing across cells. RUL from sparse complete discharges was tested by four methods and refused. The estimator was poor on cold NASA cells (12–14%), but its own consistency criterion, with a tolerance fixed on CALCE, marked those failures: 4.5% median error when it said high or medium confidence against 13.2% when it said low.

On the engineering side, the system refuses to output a number it cannot justify. Nine gates return a refusal as a value instead of an exception, the wire protocol is self-synchronising with record-level and capture-level checks, and 44 test modules and an eight-job CI pipeline pin the documentation to the artifacts that produced it. A physical rig built on a NodeMCU ESP8266 with an INA219 and an LM35 streamed 83 of 83 checksummed frames at a measured 1.000 s cadence. That validates data acquisition. It does not validate health estimation, because no discharge has yet been recorded on the rig.

Each output therefore carries a measured error band and is withheld when its evidence is missing or inconsistent, and the criterion that makes that decision was itself

validated against laboratory truth.

## 6.1 LIMITATIONS

The scope is single cells: LCO for development, an NMC/LCO blend pouch for external validation, mostly at constant current, with one real drive-cycle discharge. Packs, lithium iron phosphate and temperature-dependent capacity correction are not covered. The conventional 80% end-of-life threshold is not validated on CALCE, and the Oxford results at 80% are a conservative bound with low precision. The heuristic risk layer is not validated against measured degradation. The bench rig has been run on one cell, at rest, for under 90 s, and its two sensors have not yet operated in the same capture.

## 6.2 FUTURE WORK

In order of dependency:

1. **Hardware validation.** Capture a load step and a full constant-current discharge on the rig with all three channels real, compare the resulting capacity with an independent charger/discharger, and measure sensor accuracy against a reference meter. Then repeat on more than one cell.
2. **Protocol hardening.** Replace the XOR checksum with a CRC-8 at the same cost, which detects burst errors and byte transpositions the XOR form cannot.
3. **Removing the remaining silent default.** The serial path no longer assumes a rated capacity, but the CAN and batch paths still do. A per-vehicle registry entry is needed before the CAN path is pointed at a real vehicle.
4. **Temperature-aware correction.** Extend the ohmic correction to the charge-transfer and diffusion terms that dominate at low temperature, so that the cold-cell failure is reduced and not only flagged.
5. **Broader validation.** Test packs, lithium iron phosphate and large-scale dynamic drive cycles, and obtain data that crosses the 80% end-of-life convention for a validated result there.
6. **Persistence and operations.** Add a database behind the fleet store, structured logging and metrics, a load-test harness, and asynchronous serial ingest with backpressure.

## REFERENCES

- [1] W. Waag, C. Fleischer, and D. U. Sauer, “Critical review of the methods for monitoring of lithium-ion batteries in electric and hybrid vehicles,” *J. Power Sources*, vol. 258, pp. 321–339, 2014, doi: 10.1016/j.jpowsour.2014.02.064.
- [2] M. Bercibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, and P. Van den Bossche, “Critical review of state of health estimation methods of Li-ion batteries for real applications,” *Renew. Sustain. Energy Rev.*, vol. 56, pp. 572–587, 2016, doi: 10.1016/j.rser.2015.11.042.
- [3] J. Vetter *et al.*, “Ageing mechanisms in lithium-ion batteries,” *J. Power Sources*, vol. 147, pp. 269–281, 2005, doi: 10.1016/j.jpowsour.2005.01.006.
- [4] A. Barré, B. Deguilhem, S. Grolleau, M. Gérard, F. Suard, and D. Riu, “A review on lithium-ion battery ageing mechanisms and estimations for automotive applications,” *J. Power Sources*, vol. 241, pp. 680–689, 2013, doi: 10.1016/j.jpowsour.2013.05.040.
- [5] G. L. Plett, “Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs,” *J. Power Sources*, vol. 134, pp. 277–292, 2004, doi: 10.1016/j.jpowsour.2004.02.033.
- [6] K. A. Severson *et al.*, “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, vol. 4, no. 5, pp. 383–391, 2019, doi: 10.1038/s41560-019-0356-8.
- [7] K. Maher and N. Yerken, “Comprehensive machine learning for lithium-ion battery state-of-health estimation using group-wise cross-validation,” in *Proc. 14th Int. Conf. Renew. Energy Res. Appl. (ICRERA)*, IEEE, 2025, pp. 1465–1468, doi: 10.1109/ICRERA66237.2025.11283788.
- [8] H. H. Le and K.-A. Nguyen, “Charging phase health indicators for battery state-of-health estimation: A systematic comparison of CC, CV, and combined approaches under cross-battery validation,” arXiv:2607.23482, 2026.
- [9] I. Bloom *et al.*, “Differential voltage analyses of high-power, lithium-ion cells,” *J. Power Sources*, vol. 139, pp. 295–303, 2005, doi: 10.1016/j.jpowsour.2004.07.021.

- [10] M. Dubarry, C. Truchot, and B. Y. Liaw, “Synthesize battery degradation modes via a diagnostic and prognostic model,” *J. Power Sources*, vol. 219, pp. 204–216, 2012, doi: 10.1016/j.jpowsour.2012.07.016.
- [11] C. Weng, Y. Cui, J. Sun, and H. Peng, “On-board state of health monitoring of lithium-ion batteries using incremental capacity analysis with support vector regression,” *J. Power Sources*, vol. 235, pp. 36–44, 2013, doi: 10.1016/j.jpowsour.2013.02.012.
- [12] C. R. Birkl, M. R. Roberts, E. McTurk, P. G. Bruce, and D. A. Howey, “Degradation diagnostics for lithium ion cells,” *J. Power Sources*, vol. 341, pp. 373–386, 2017, doi: 10.1016/j.jpowsour.2016.12.011.
- [13] J. Tian, R. Xiong, W. Shen, J. Lu, and X.-G. Yang, “Deep neural network battery charging curve prediction using 30 points collected in 10 min,” *Joule*, vol. 5, pp. 1521–1534, 2021, doi: 10.1016/j.joule.2021.05.012.
- [14] X. Hu, L. Xu, X. Lin, and M. Pecht, “Battery lifetime prognostics,” *Joule*, vol. 4, pp. 310–346, 2020, doi: 10.1016/j.joule.2019.11.018.
- [15] B. Saha and K. Goebel, “Battery data set,” NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2007.
- [16] Y. Liu, S. Saxena, M. Pecht, *et al.*, “CALCE CS2 and CX2 prismatic cell cycling series,” Center for Advanced Life Cycle Engineering, University of Maryland, College Park, MD, USA.
- [17] D. Howey and C. Birkl, “Oxford battery degradation dataset 1,” University of Oxford, 2017, doi: 10.5287/bodleian:KO2kdmYGg.
- [18] C. R. Birkl, “Diagnosis and prognosis of degradation in lithium-ion batteries,” Ph.D. dissertation, Dept. Eng. Sci., Univ. Oxford, Oxford, U.K., 2017.

## APPENDIX A

### Provenance of Every Reported Number

Every number in this report is regenerated by a named script from committed artifacts. Table A.1 maps each results section to the artifact under `reports/metrics/` and the script under `scripts/` that produced it.

Table A.1: Artifacts and Scripts Behind Each Results Section

| Section                | Artifact                                          | Script                                            |
|------------------------|---------------------------------------------------|---------------------------------------------------|
| 5.1                    | benchmark_results.csv,<br>ablation/               | run_benchmark_study.py,<br>run_ablation_study.py  |
| 5.2                    | calce_field_soh/                                  | run_field_soh_study.py                            |
| 5.3                    | calce_partial_soh/                                | run_partial_soh_study.py                          |
| 5.4                    | calce_resistance/                                 | run_resistance_study.py                           |
| 5.5                    | calce_rul_horizon_<br>early_ref/, error_bands.csv | run_rul_horizon_study.py,<br>build_error_bands.py |
| 5.6                    | calce_field_rul/                                  | run_field_rul_study.py                            |
| 5.7                    | calce_sufficiency/                                | run_sufficiency_study.py                          |
| 5.8                    | oxford/                                           | run_oxford_study.py,<br>run_oxford_drive_cycle.py |
| 5.9                    | cross_dataset/                                    | run_cross_dataset_study.py                        |
| 5.10                   | data/interim/rig_*.txt                            | run_serial_demo.py,<br>panel_demo.py              |
| 5.11                   | Console output of the report card                 | health_report.py                                  |
| Figs. 5.1,<br>5.2, 5.3 | reports/figures/<br>results_*.png                 | export_results.py                                 |

## APPENDIX B

### Reproducing the System and the Results

The repository is <https://github.com/Navvu-gityhub/Behavior-Aware-BMS>. The commands below run on a fresh clone with no hardware, no database, no API keys and no dataset download.

```
git clone \
    https://github.com/Navvu-gityhub/Behavior-Aware-BMS.git
cd Behavior-Aware-BMS
pip install -r requirements-dev.txt

python main.py # end-to-end run
python scripts/run_serial_demo.py # emulated rig
python scripts/health_report.py --serial \
    data/interim/rig_stage_b_voltage_verified.txt
python -m pytest tests/ -q # the test suite
docker compose up # API, gateway, client
```

With the CALCE data downloaded, the report card of a real cell as it read at a given cycle is printed by

```
python scripts/health_report.py --calce \
    data/raw/calce/CS2/Type2/CS2_35.zip --upto-cycle 90
```

The raw CALCE and NASA data are not redistributed, and download locations are given in the repository.